HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

TRƯỜNG ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Kiểm chứng và kiểm tra vi mạch

PGS. TS. Nguyễn Đức Minh
Khoa Điện tử, Trường Điện-Điện tử
Đại học Bách Khoa Hà Nội

ONE LOVE. ONE FUTURE.

Giới thiệu

- Giảng viên
  - Nguyễn Đức Minh
  - Email: minh.nguyenduc1@hust.edu.vn
  - Văn phòng: C7-811/C7-616
- Trợ giảng
  - Trần Tùng Dương
  - Trương Đại Dương
- Lớp học
  - Chiều Thứ 7 hàng tuần
- Đánh giá
  - Nhóm 3 người
  - Làm project cuối kỳ 50% - Thi vấn đáp
  - Các bài quiz + lab: 40%
  - Điểm danh: 10%

Mục tiêu buổi học - Tuần 06

1. Kiểu dữ liệu trong SV

2. Giới thiệu về UVM, OVM và VMM

3. Cấu trúc một testbench theo UVM

Kiểu dữ liệu trong Systemverilog

Khái quát:

- SystemVerilog mở rộng từ Verilog với các kiểu dữ liệu mới, nhằm hỗ trợ tốt hơn cho quá trình thiết kế và kiểm thử hệ thống số.
- Các kiểu dữ liệu trong SystemVerilog có thể được chia thành hai nhóm chính: **kiểu dữ liệu 4 trạng thái** và **kiểu dữ liệu 2 trạng thái**.
- Ngoài ra, SystemVerilog còn hỗ trợ:
  - Các mảng (arrays)
  - cấu trúc (structures)
  - kiểu liệt kê (enumerated types)
  - Các kiểu dữ liệu động như hàng đợi (queues) và mảng liên kết (associative arrays).

Kiểu dữ liệu trong Systemverilog

1. Kiểu dữ liệu cơ bản

1.1 Kiểu dữ liệu hai trạng thái:

- **Bit, byte, shortint, int, longint**: Đây là các kiểu dữ liệu chỉ có hai trạng thái 0 và 1. Các giá trị không xác định x và z sẽ được ánh xạ thành 0 hoặc 1.
- Ví dụ:

```
bit [7:0] byte_val;     // Kiểu byte
int count;              // Kiểu int 32-bit có dấu
longint large_value;    // Kiểu longint 64-bit có dấu
```

1.2 Kiểu dữ liệu bốn trạng thái:

- **Logic**: Là kiểu dữ liệu chính trong SystemVerilog, thay thế cho reg và wire trong Verilog. Kiểu logic có thể chứa 4 trạng thái: 0, 1, z (trạng thái cao trở kháng), và x (trạng thái không xác định).
- Ví dụ:

```
logic [31:0] data;  // Biến 32-bit có 4 trạng thái
logic resetN;       // Biến 1-bit
```

2. Mảng (Array)

2.1 Mảng cố định (Fixed-size array):

- Mảng cố định được khai báo với kích thước đã xác định trước.
- SV hỗ trợ cả mảng một chiều và mảng đa chiều. Mảng cố định có thể là **packed** hoặc **unpacked**, và kích thước phải được xác định tại thời điểm biên dịch. Ví dụ:

```
logic [7:0] fixed_array [0:3];  // Mảng 4 phần tử, mỗi phần tử 8-bit
logic [7:0] multi_array [0:3][0:7];  // Mảng 2 chiều
```

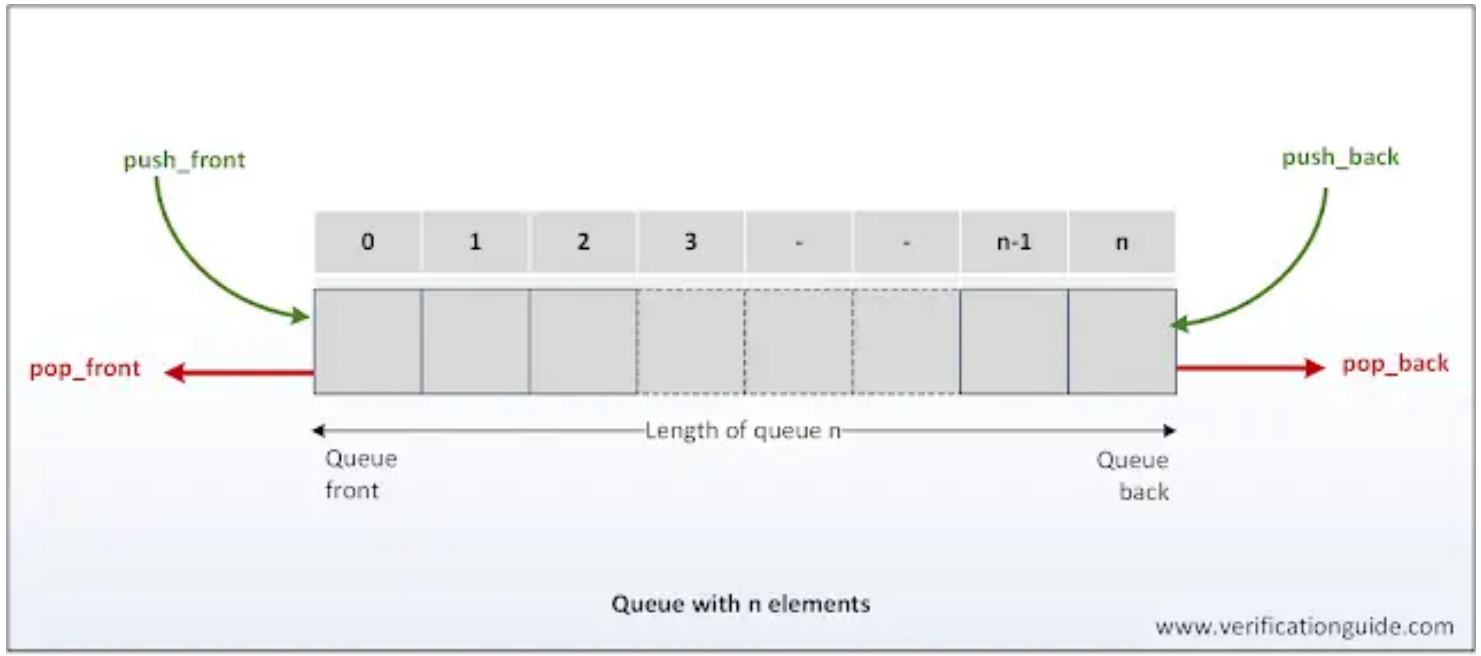
2.2 Mảng động (Dynamic array):

- Mảng động có kích thước được xác định tại thời gian chạy và có thể thay đổi kích thước trong quá trình mô phỏng. Chúng được khai báo mà không có kích thước cụ thể và sử dụng toán tử new[] để phân bổ bộ nhớ. Ví dụ:

```
int dynamic_array[];   // Khai báo mảng động
initial begin
    dynamic_array = new[10];  // Phân bổ 10 phần tử
end
```

3. Hàng đợi (Queue)

- Hàng đợi là một loại **mảng** có thể thêm và xóa các phần tử một cách dễ dàng từ cả hai đầu.
- Các thao tác phổ biến như *push_back*, *pop_front*, *insert*, *delete* giúp quản lý hàng đợi linh hoạt.
- Ví dụ:

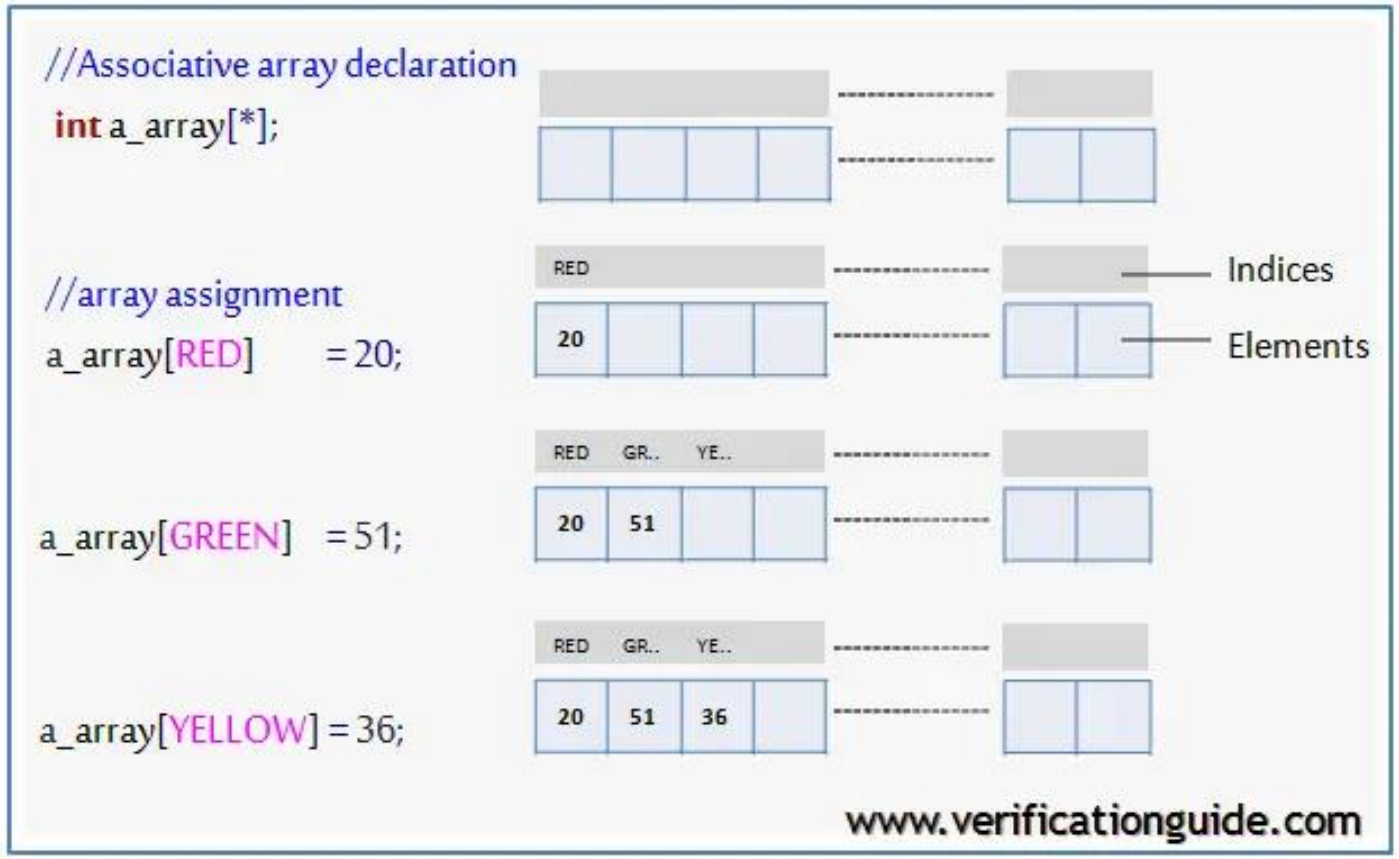



```
int queue[$] = {1, 2, 3};   // Hàng đợi có 3 phần tử
queue.push_back(4);         // Thêm phần tử vào cuối hàng đợi
queue.pop_front();          // Lấy và xóa phần tử đầu tiên
```

4. Mảng liên kết (Associative array)

- Mảng liên kết sử dụng các chỉ số không liên tục và có thể được truy cập bằng số nguyên hoặc chuỗi. Loại mảng này rất hữu ích khi làm việc với dữ liệu thưa thớt.
- Ví dụ:

```
logic [63:0] assoc_mem[*];  // Mảng liên kết
assoc_mem[1] = 64'hA1;      // Truy cập thông qua chỉ số
```

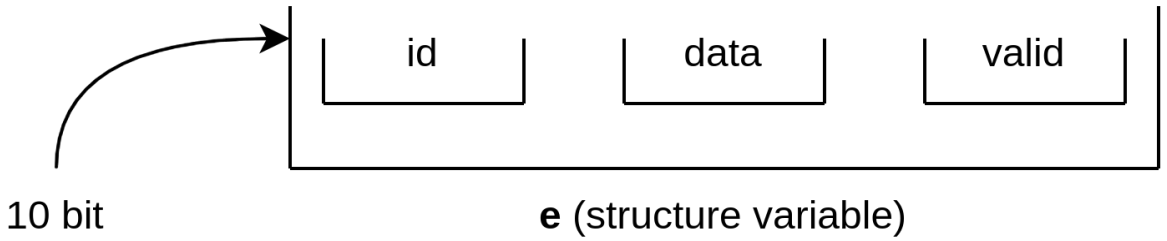


5. Cấu trúc (Structure)

- **Structures** cho phép nhóm nhiều biến lại với nhau để dễ quản lý và sử dụng. Các cấu trúc này có thể được truy cập toàn bộ hoặc từng thành phần riêng lẻ.
- **Ví dụ**:

```
struct {
    int id;
    logic [7:0] data;
    bit valid;
} packet;
```

- **Phân loại structures:**
  - Packed structure
  - Unpacked structure



Kiểu dữ liệu trong Systemverilog

5. Cấu trúc (Structure)

- **Packed Structures** được khai báo dưới dạng packed để đảm bảo các thành phần của nó được lưu trữ liên tục trong bộ nhớ. Điều này cho phép thực hiện các phép toán logic trên toàn bộ cấu trúc.
- **Ví dụ:**

```
struct packed {
    logic [3:0] tag;
    logic [7:0] data; } packed_data;
 packed_data = 'b11001010;
```

- **Unpacked Structures** là kiểu mặc định của struct, được sử dụng để định nghĩa một nhóm các biến có các kiểu dữ liệu khác nhau
- **Ví dụ:**

```
struct {
    logic [7:0]   data1;
    logic [15:0] data2;
    bit              enable; }
myStruct;
```

Kiểu dữ liệu trong Systemverilog

6. Kiểu liệt kê (Enumerated types)

- **Kiểu liệt kê (enum)** trong SystemVerilog cho phép định nghĩa các biến chỉ nhận một tập hợp giá trị cụ thể. Điều này giúp mã trở nên dễ đọc và dễ bảo trì hơn.

- **Ví dụ:**

```
enum {RED, GREEN, BLUE} color;
color = GREEN;
```

- Kiểu liệt kê cũng có thể được gán giá trị cụ thể cho mỗi nhãn:

- **Ví dụ:**

```
enum logic [1:0] {
IDLE = 2'b00,
RUN = 2'b01,
STOP = 2'b10}
state;
```

7. Phương pháp mảng (Array methods)

- SystemVerilog cung cấp nhiều phương pháp tích hợp để thao tác với mảng, bao gồm các phương thức như find, sort, sum, và shuffle.
- **Ví dụ phương thức find:**

```
int array[] = {1, 3, 5, 7};
int found = array.find_first with
(item > 3);
```

- **Ví dụ phương thức sum**

```
int array[] = {1, 2, 3, 4};
int total = array.sum();
$display("Total Sum: %0d", total);
// Output sẽ là 10
```

- Kiểu liệt kê cũng có thể được gán giá trị cụ thể cho mỗi nhãn:
- **Ví dụ phương thức sort:**

```
int array[] = {10, 4, 15, 7};
 // Sắp xếp mảng theo thứ tự tăng dần
array.sort();
$display("Sorted: %p", array);
// Output sẽ là {4, 7, 10, 15}
```

- **Ví dụ phương thức shuffle**

```
int array[] = {1, 2, 3, 4};
$display("Shuffled Array: %p", array);
// Output sẽ là mảng với thứ tự ngẫu
nhiên, ví dụ {3, 1, 4, 2}
```

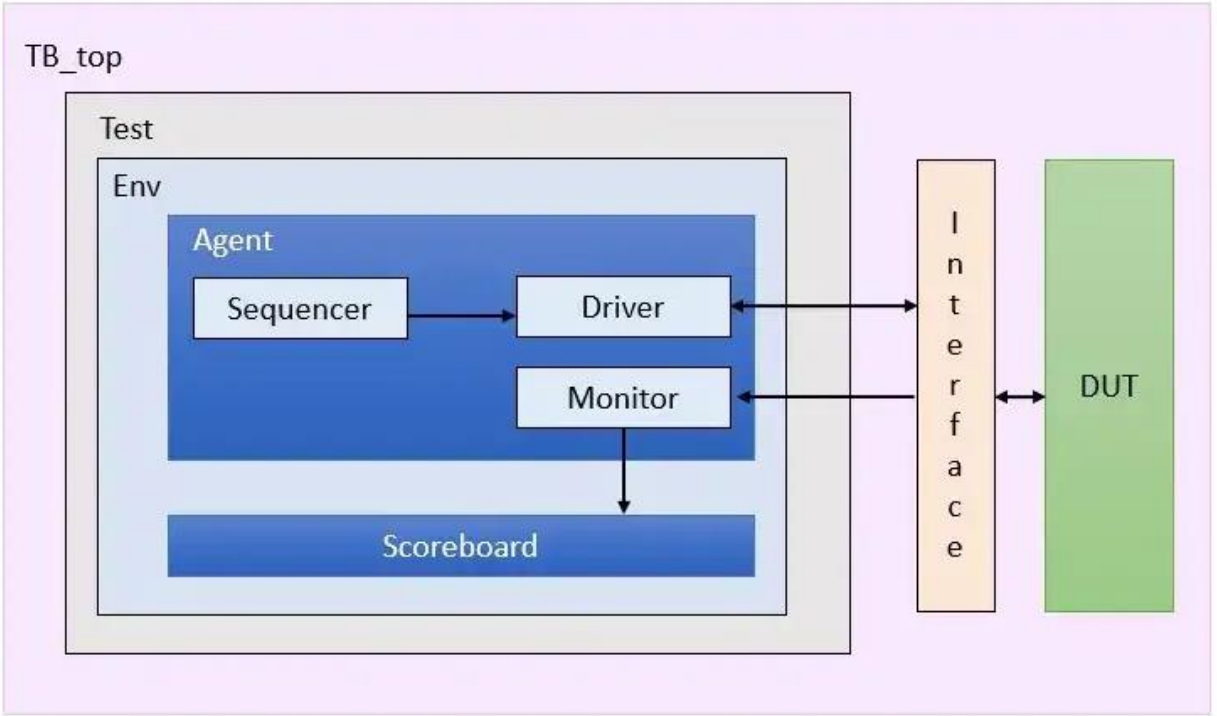
I. Giới thiệu chung về UVM, OVM, VMM

1. Định nghĩa UVM

- Viết tắt của Universal Verification Methodology, (2011) và là thành công sau nền tảng là OVM.
- Là một phương pháp chuẩn hóa để kiểm chứng các thiết kế số và hệ thống trên chip (SoCs), xây dựng bằng systemverilog.

2. Vai trò UVM

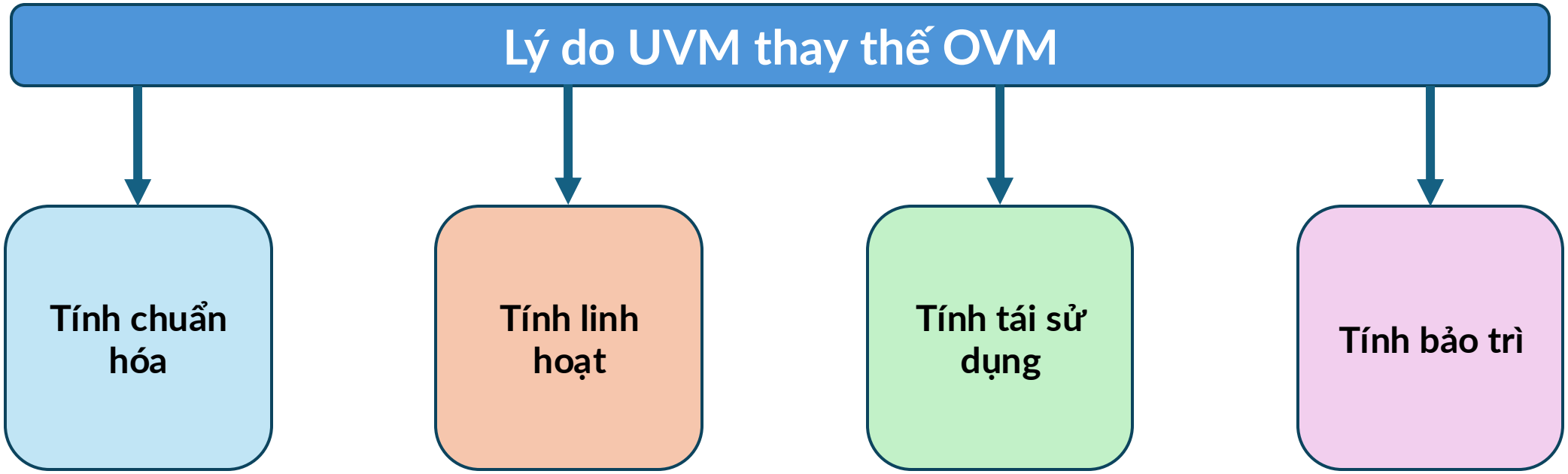
- Cung cấp framework để tạo ra các thành phần testbench module, khả năng tái sử dụng được và dễ dàng tích hợp vào quy trình kiểm thử thiết kế.
- Tích hợp các methodology để chạy mô phỏng và phân tích kết quả.



Ví dụ minh họa cấu trúc của một UVM

I. Giới thiệu chung về UVM, OVM, VMM

- *OVM (Open Verification Methodology) được giới thiệu vào năm 2008 là một phương pháp xác minh mã nguồn mở cho các thiết kế kỹ thuật số và hệ thống trên chip (SoCs), và được dựa trên SystemVerilog.*



OVM mã nguồn mở nhưng thiếu một tiêu chuẩn chính thức -> UVM được thiết kế với sự chuẩn hóa toàn cầu đáp ứng sự thống nhất.

OVM thiết kế cho mô hình hóa mức giao dịch (TLM), không hỗ trợ mô hình hóa RTL -> UVM hỗ trợ cả TLM, RTL và nhiều kịch bản khác linh hoạt.

OVM có bộ các lớp và thành phần định nghĩa sẵn nhưng không phải module nên khó tái sử dụng -> UVM chứa các module các thành phần rõ ràng trong tb

OVM không phải lúc nào cũng dễ bảo trì hoặc cập nhật -> Tính module hóa rõ ràng khiến UVM dễ bảo trì và cập nhật

I. Giới thiệu chung về UVM

- *VMM (Verification Methodology Manual) là một phương pháp luận kiểm thử bởi Synopsys (2005), nhằm cung cấp một hướng dẫn tiêu chuẩn cho việc xác minh các thiết kế kỹ thuật số và SoCs, dựa trên SystemVerilog và cung cấp một framework và các lớp cơ bản để xây dựng testbench.*

❑ Lý do UVM được sử dụng phổ biến hơn VMM:

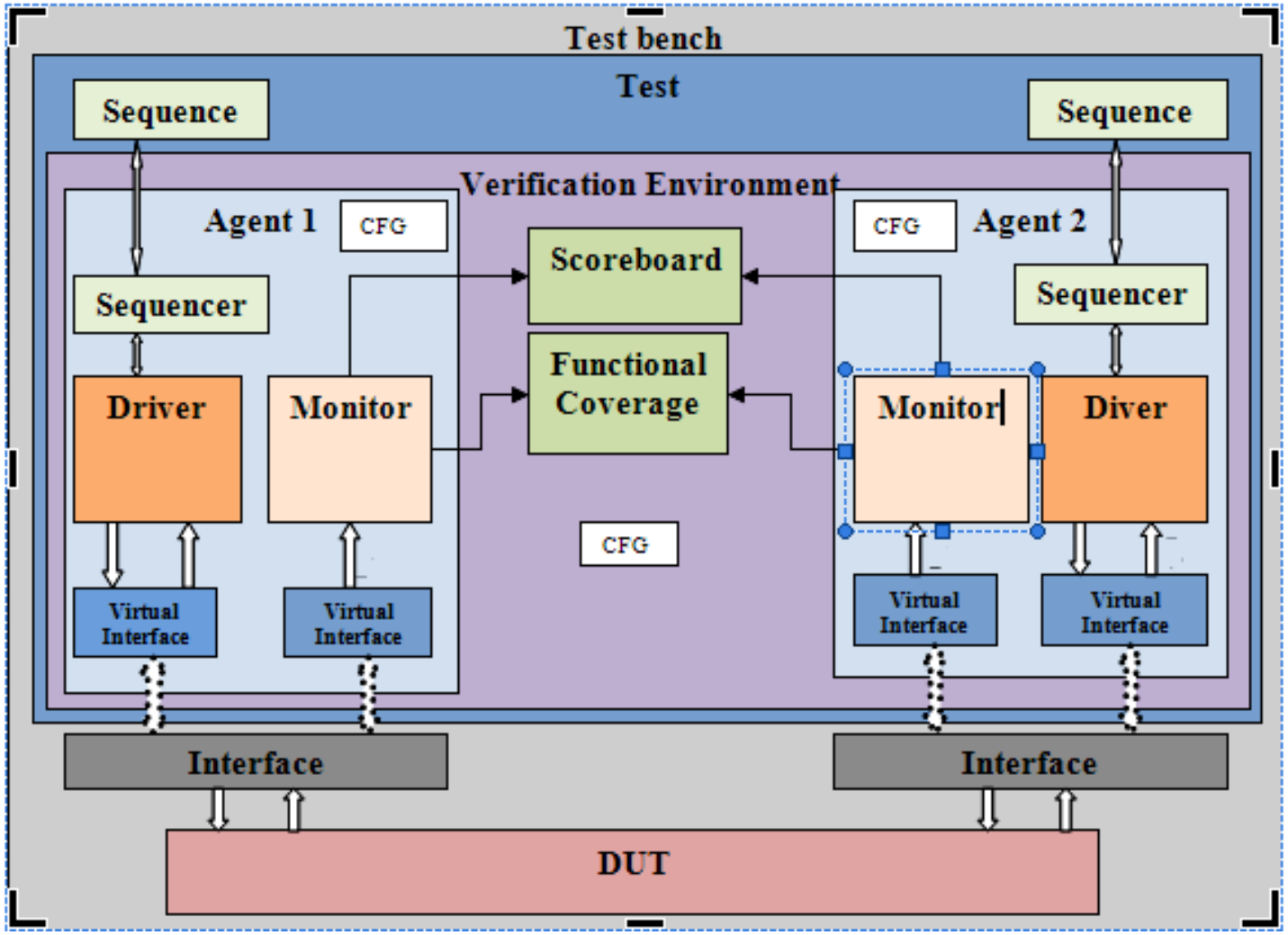
1. VMM do Synopsys phát triển và không phải là một tiêu chuẩn mở, do đó VMM hạn chế **tính tương thích** với các công cụ và môi trường từ các nhà cung cấp khác so với UVM.

2. VMM có các công cụ hỗ trợ tốt cho mô hình hóa và kiểm tra các hệ thống RTL, nhưng UVM **linh hoạt hơn** nhờ khả năng hỗ trợ cả TLM và RTL

3. VMM cũng cung cấp các thành phần cơ bản, nhưng nó không được thiết kế tối ưu cho việc **tái sử dụng** ở quy mô rộng như UVM.

2. Cấu trúc một testbench theo UVM

- UVM bao gồm các lớp và các phương thức/module được định nghĩa sẵn và có khả năng tái sử dụng.
- Ví dụ bên cạnh cho thấy cách một môi trường kiểm thử điển hình được xây dựng bằng cách mở rộng các lớp UVM có sẵn (màu vàng). Các thành phần này đã có sẵn và có thể kết nối với nhau, xử lý các gói dữ liệu và làm việc đồng bộ với các thành phần khác.
- Các khái niệm tiếp theo cần quan tâm:
  - Transactions
  - Sequencer
  - Driver
  - Monitor
  - Scoreboard

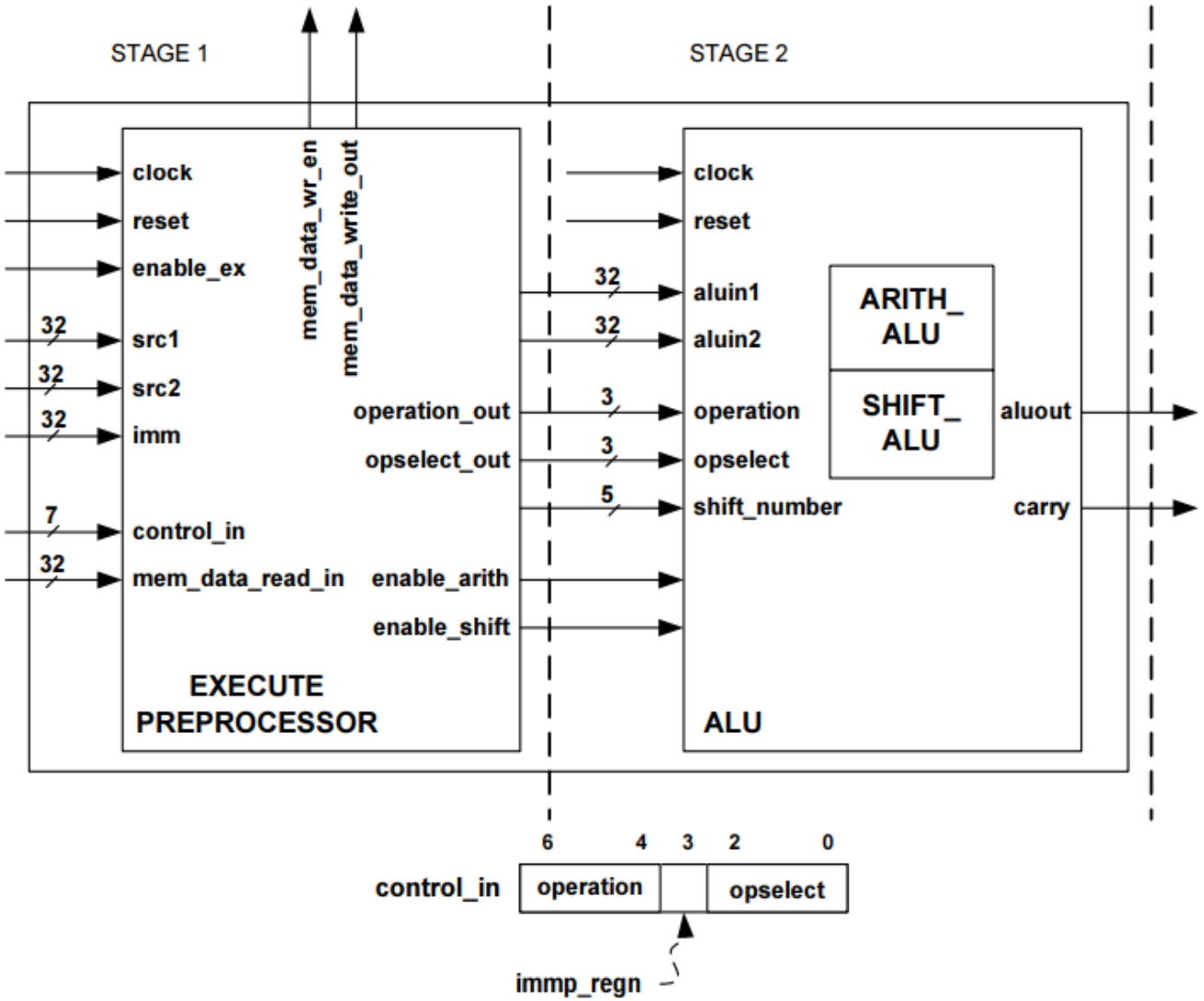


ALU Specification

- Đơn vị logic số học (ALU) là thành phần chính của bộ xử lý chịu trách nhiệm thực hiện các phép toán số học và logic.
- Thiết kế ALU dựa trên quy trình hai giai đoạn bao gồm Execute Pre-processor (Bộ xử lý trước thực thi) và ALU.
- ALU xử lý các phép toán số học và logic được điều khiển bởi các đầu vào như opselect và phép toán.

ALU Specification

- **Bước tiền xử lý** thực hiện chức năng chuẩn bị các đầu vào và điều khiển cho **ALU** sử dụng thông tin nhận được từ đầu vào
- **ALU** sau đó thực thi các phép toán liên quan đến dữ liệu đầu vào được điều khiển bởi các giá trị **opselect**, **operation**, **shift_number**, **enable_arith** và **enable_shift**
- Kết quả xử lý cuối cùng được tạo ra dưới dạng **aluout** và **carry**



ALU Specification

Signal name	Width	I/O	Description
src1	32	Input	Toán hạng nguồn thứ nhất
src2	32	Input	Toán hạng nguồn thứ hai
imm	32	Input	Giá trị tức thời
mem_data_read_in	32	Input	Dữ liệu đọc từ bộ nhớ
control_in	7	Input	Tín hiệu điều khiển
aluout	32	Output	Kết quả của ALU
carry	1	Output	Bit nhớ

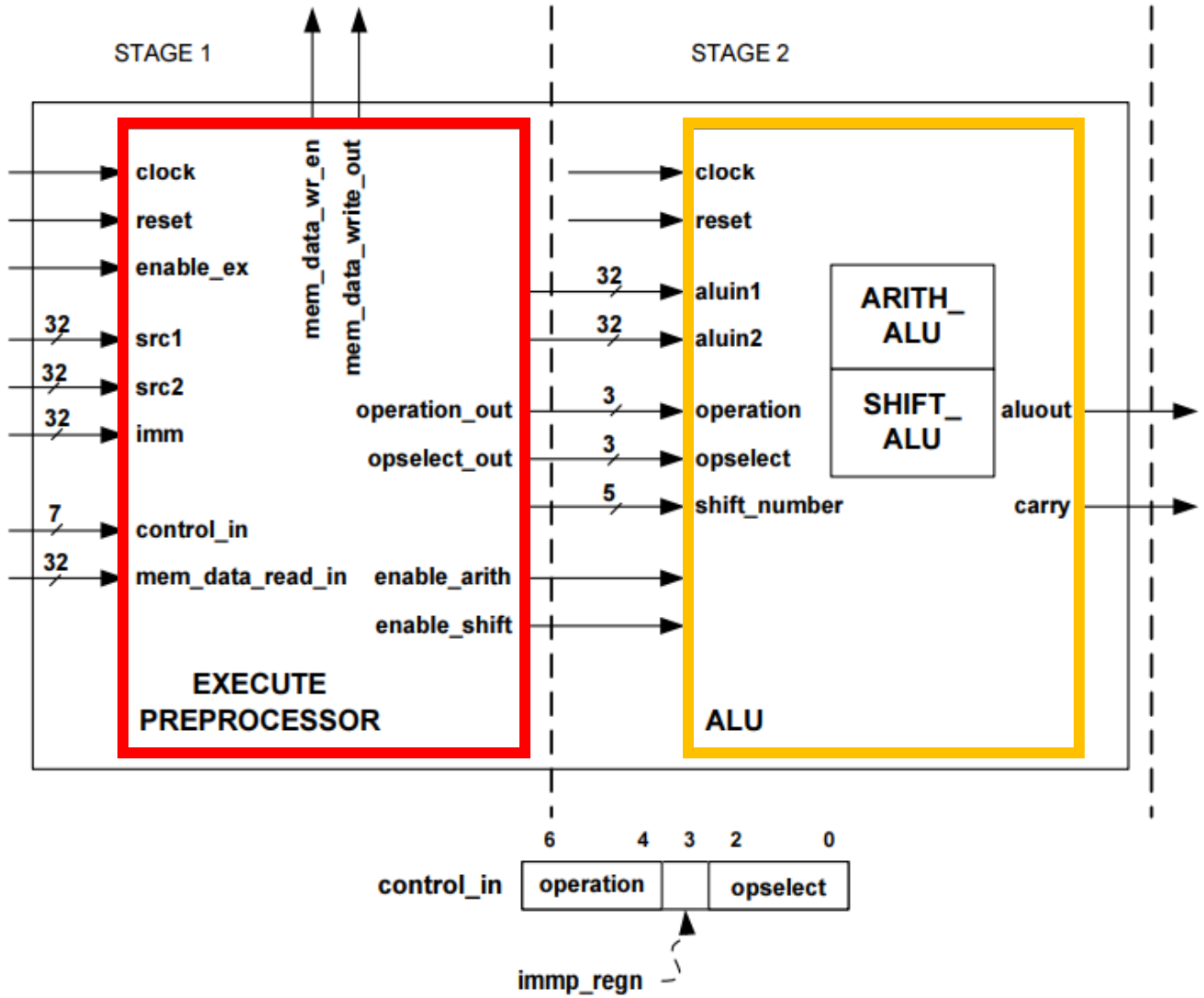
ALU Specification

- Control Signals:
  - opselect: Selects the operation to be performed by the ALU.
  - operation: Specifies the actual operation (ADD, SUB, AND, OR, etc.).
  - shift_number: Defines the shift amount for shift operations.

- Operations:
  - Arithmetic: ADD, SUB, NOT, AND, OR, XOR.
  - Shifting: Logical and arithmetic shifts (left and right).
  - Memory operations: LOADBYTE, LOADHALF, LOADWORD.

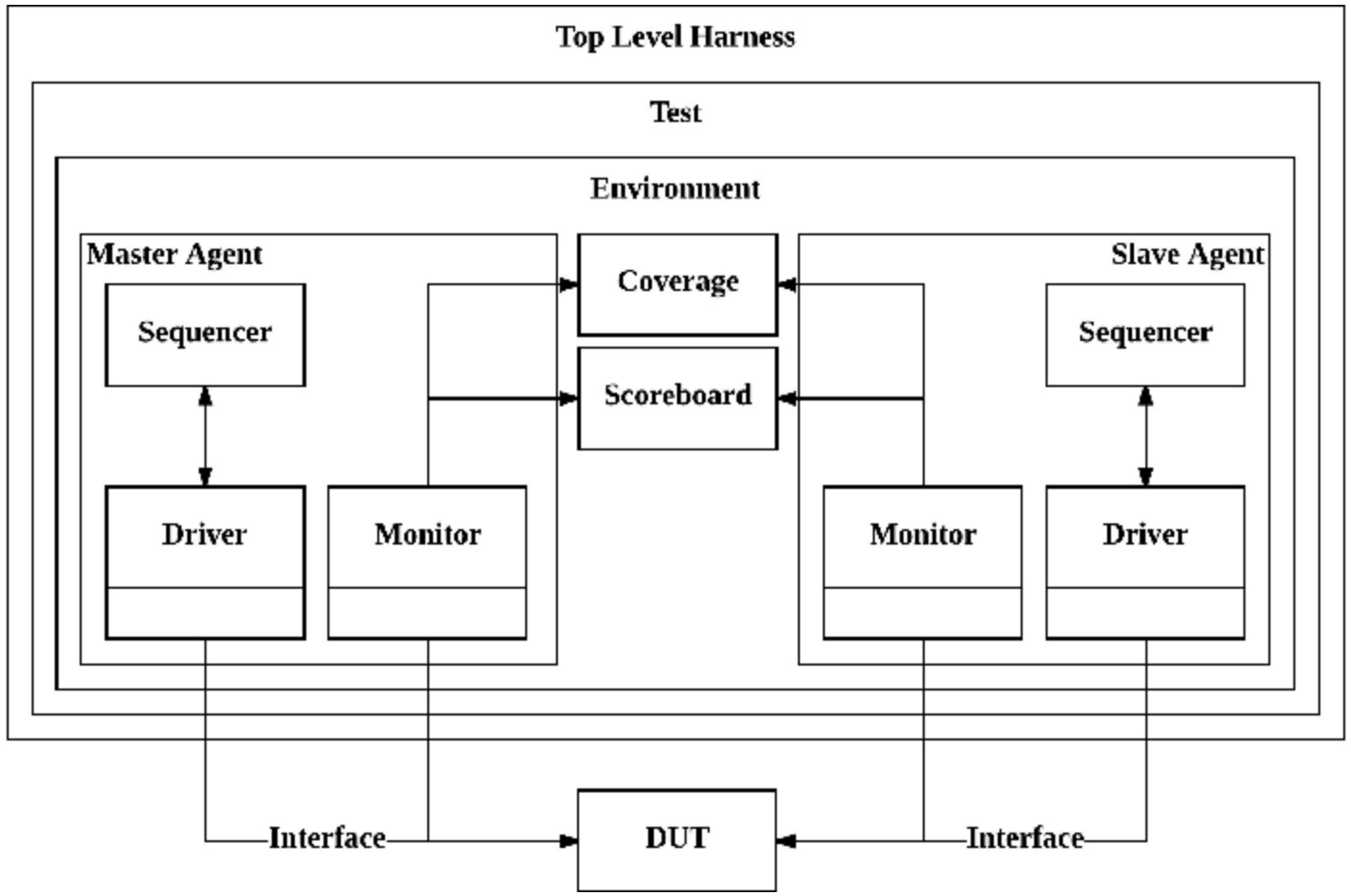
ALU Specification

- **Stage 1:**
  - Chuẩn bị các toán hạng và tín hiệu điều khiển cho giai đoạn ALU.
  - Đầu ra là các tín hiệu trung gian như aluin1, aluin2, enable_arith, và enable_shift.
- **Stage 2:**
  - Thực hiện các phép toán số học hoặc logic trên aluin1 và aluin2.
  - Đầu ra là kết quả (aluout) và cờ nhớ (carry).



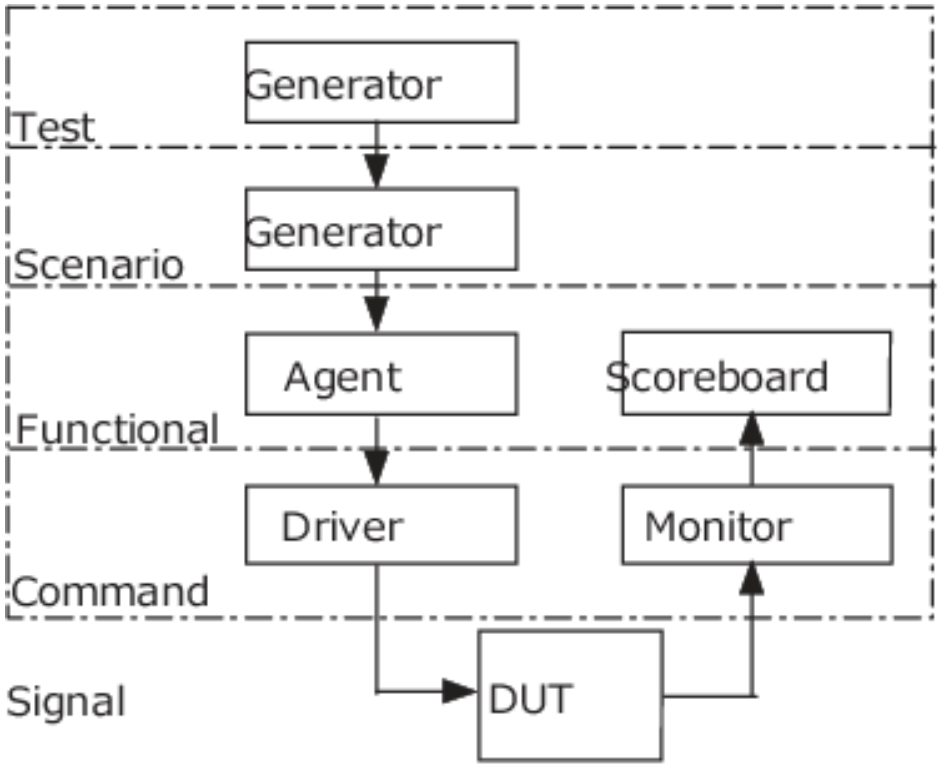
Tổng quan về kiến trúc testbench

- **Top-Level Harness**: Đóng vai trò là lớp bao bọc cho toàn bộ testbench.
- **Test Layer**: Chứa các kích thích (stimulus) và môi trường cho quá trình kiểm thử.
- **Environment**: Bao gồm các tác nhân (agents) cho master và slave, bảng ghi điểm (scoreboard), và thu thập độ phủ (coverage collection).
- **Master & Slave Agents**: Mỗi tác nhân đều bao gồm một sequencer, driver, và monitor để tạo ra và thu nhận các giao dịch.
- **DUT** (Thiết bị đang kiểm thử): ALU thực hiện các phép toán số học và logic.
- **Interfaces**: Định nghĩa các tương tác tín hiệu giữa testbench và DUT.



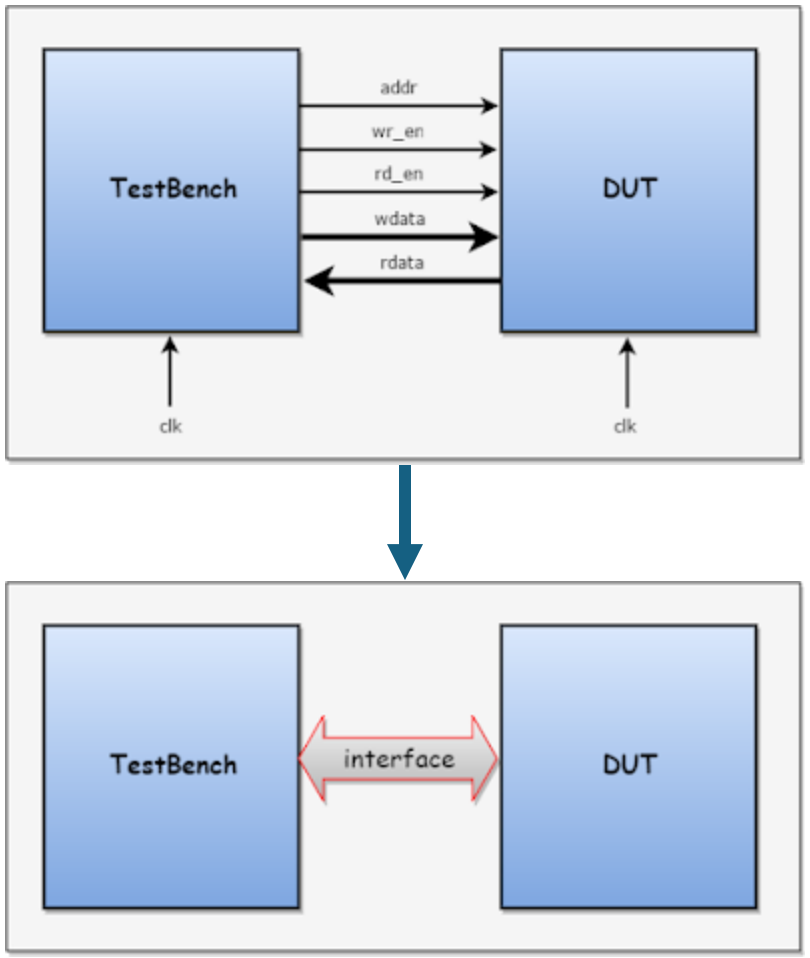
Tổng quan về kiến trúc testbench

- Cấu trúc testbench phân theo lớp đóng gói thông tin
  - **Packet**: Đưa gói tin vào DUT từ phía phát. Áp đặt các constraint đối với việc ngẫu nhiên hóa các đầu vào khác nhau cho DUT.
  - **Generator**: Được sử dụng để tạo đầu vào liên quan đến DUT bằng cách sử dụng các constraint được áp đặt trong lớp Packet.
  - **Driver**: Nhận các gói tin và chuyển đổi nó thành các tín hiệu interface
  - **Monitor**: Cung cấp các gói tin cho Scoreboard
  - **Scoreboard**: Nhận gói tin từ monitor, tính toán đầu ra rồi so sánh với thiết kế



1. Chức năng của Interface

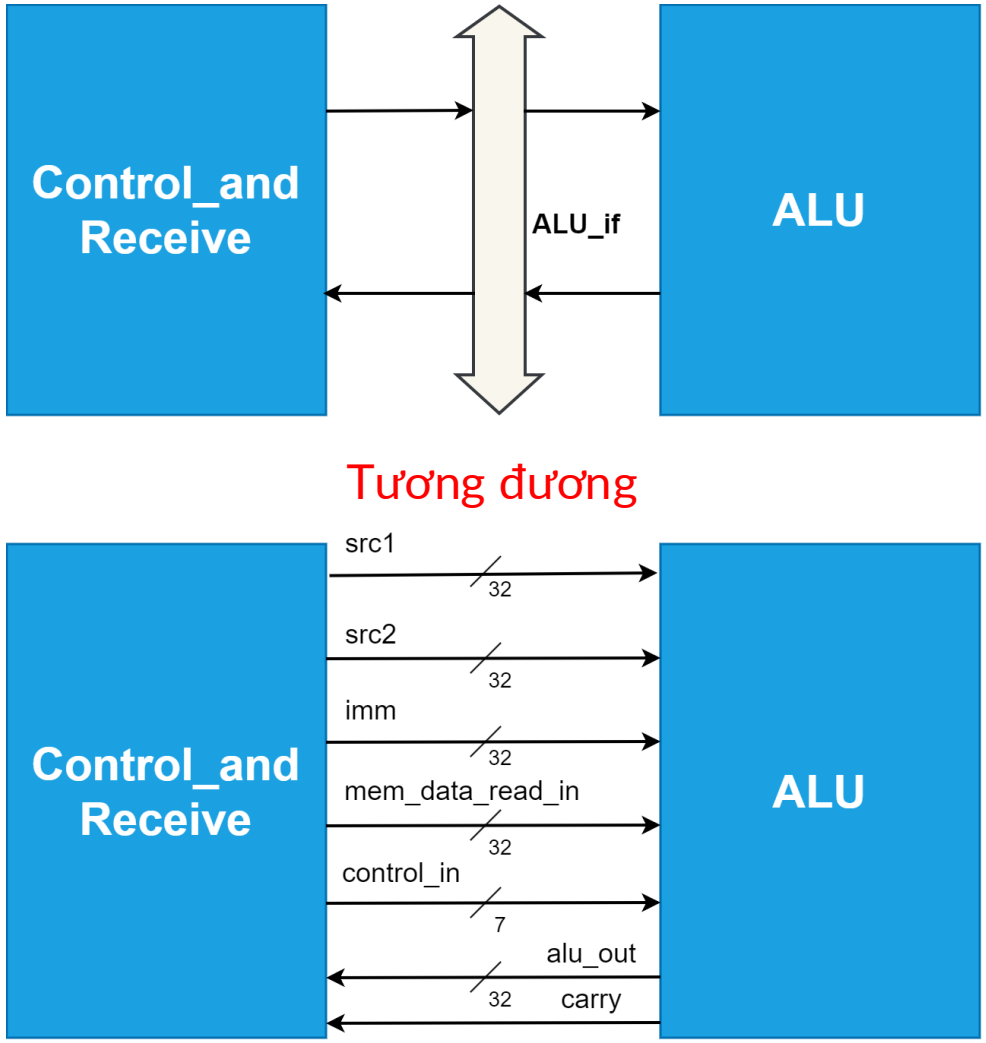
- Đóng gói các tín hiệu: Interface gom nhóm các tín hiệu liên quan lại với nhau, giúp đơn giản hóa việc giao tiếp giữa các module và các testbench.
- Giảm sự trùng lặp: Thay vì kết nối từng tín hiệu riêng lẻ, interface kết nối nhiều tín hiệu gọn gàng và dễ bảo trì hơn.
- Dễ tái sử dụng: Interface giúp việc tái sử dụng các nhóm tín hiệu trên các thành phần khác nhau của testbench trở nên dễ dàng hơn.



2. Đóng gói các tín hiệu trong Interface

- Interface nhóm các tín hiệu thành một cấu trúc duy nhất, giúp dễ dàng xử lý các tập hợp tín hiệu liên quan, chẳng hạn như tín hiệu cho một ALU hoặc hệ thống bộ nhớ.
- Đóng gói đảm bảo rằng tất cả các tín hiệu cần thiết cho việc giao tiếp được xử lý như một khối, giảm khả năng bỏ sót các kết nối.

```
interface alu_if (input logic clock, reset, enable_ex);
    logic [31:0] src1, src2, imm, mem_data_read_in;
    logic [6:0] control_in;
    logic [31:0] aluout;
    logic carry;
endinterface
```



3. Modport trong interface

Xác định hướng (input, output, inout) của các tín hiệu cho các người dùng khác nhau (ví dụ: DUT, driver, monitor)

Kiểm soát cách các thành phần khác nhau tương tác với interface bằng cách chỉ rõ tín hiệu nào có thể đọc hoặc ghi bởi mỗi user.

```
interface alu_if(input logic clock, reset, enable_ex);
  logic [31:0] src1, src2, imm, mem_data_read_in;
  logic [6:0] control_in;
  logic [31:0] aluout;
  logic carry;

  modport DUT_mp (input clock, reset, enable_ex, src1,
src2, imm, control_in, output aluout, carry);
  modport DRV_mp (input clock, reset, enable_ex, src1,
src2, imm, control_in);
  modport MON_mp (input clock, reset, enable_ex, aluout,
carry);
endinterface
```

1. Modport DUT_mp: clock, reset, enable_ex, src1, src2, imm, và control_in được khai báo là input; aluout và carry là output.
2. Modport DRV_mp: Chỉ có clock, reset, enable_ex, src1, src2, imm, và control_in được khai báo là input; không có tín hiệu nào được khai báo là output, nghĩa là DRV_mp không có quyền ghi vào các tín hiệu đầu ra như aluout và carry.
3. Modport MON_mp: clock, reset, enable_ex là input, các tín hiệu aluout và carry được khai báo là input trong modport này, vì monitor chỉ cần đọc các tín hiệu này mà không cần ghi.

3. Clocking block trong interface

- Clocking block trong interface là các cấu trúc đặc biệt được sử dụng để đồng bộ hóa các tín hiệu dựa trên các sự kiện của xung nhịp.
- Chúng xác định các tín hiệu đầu vào và đầu ra theo xung nhịp, đảm bảo thời gian chính xác trong quá trình lấy mẫu và điều khiển tín hiệu.

```
interface alu_if(input logic clock, reset);
  clocking cb @(posedge clock);
    input src1, src2, imm, control_in;
    output aluout, carry;
  endclocking
endinterface
```

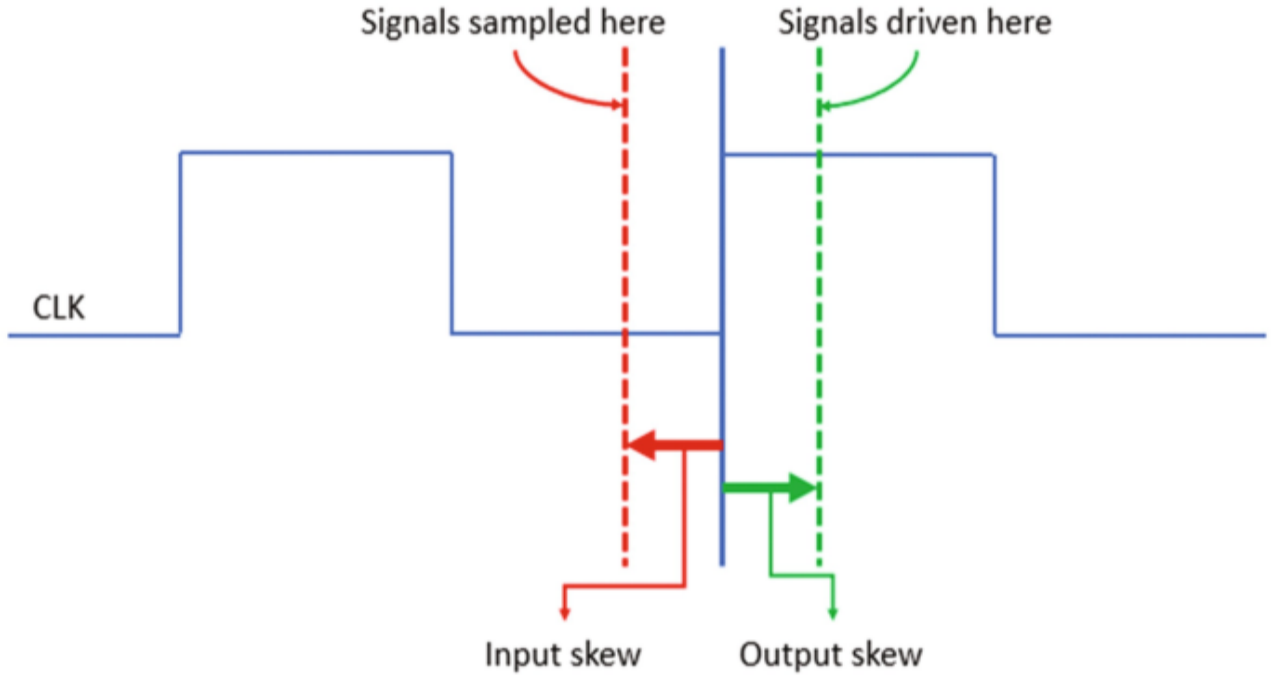
Khái niệm input skew, output skew:

- **Input skew**: Thời gian trễ khi lấy mẫu tín hiệu đầu vào, so với cạnh của clock. Điều này cho phép tín hiệu đầu vào được lấy mẫu trước hoặc sau một khoảng thời gian nhất định so với cạnh xung clock. Trong ảnh, tín hiệu đầu vào được lấy mẫu sớm hơn một chút so với cạnh xung clock.
- **Output skew**: Thời gian trễ khi điều khiển tín hiệu đầu ra, cho phép tín hiệu đầu ra được phát ra muộn hơn một chút so với cạnh clock. Trong ảnh, tín hiệu đầu ra được điều khiển ngay sau cạnh xung clock.

3. Clocking block trong interface: input skew & output skew

```
interface alu_if(input logic clock, reset);
    clocking cb @(posedge clock);
        input #2ns src1, src2, imm, control_in;
        output #1ns aluout, carry;
    endclocking
endinterface
```

- Trong cú pháp **clocking block**, bạn có thể định nghĩa **input skew** và **output skew** để điều chỉnh thời điểm tín hiệu được lấy mẫu (sample) hoặc được điều khiển (drive) dựa trên xung nhịp (**CLK**).



3. Clocking block trong interface: Vai trò

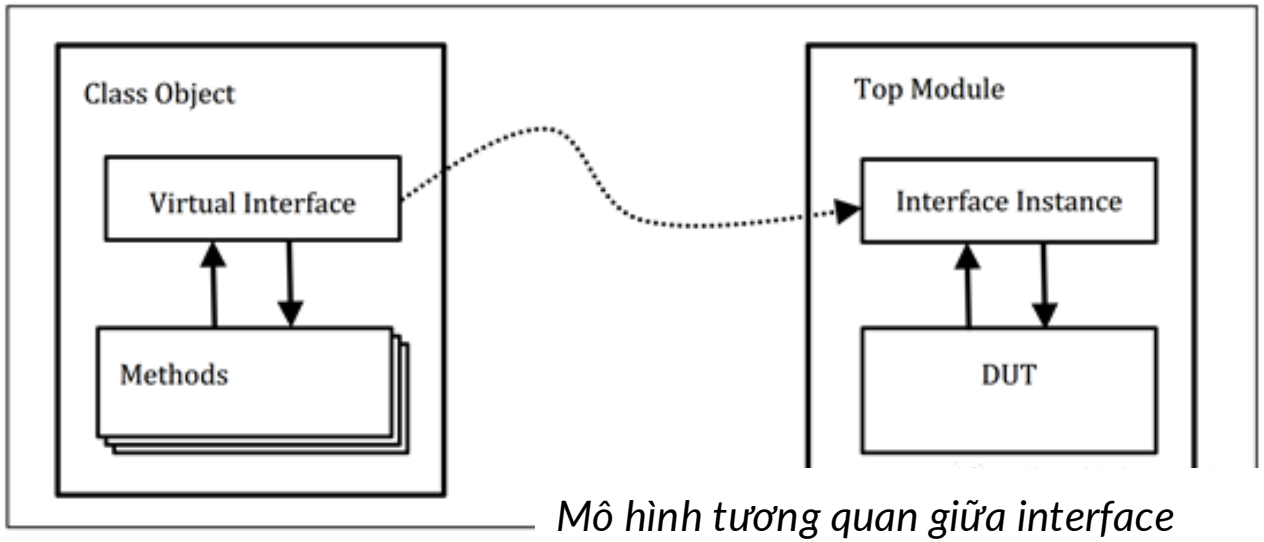
1. Đồng bộ hóa: Đảm bảo rằng các tín hiệu đầu vào và đầu ra được lấy mẫu hoặc điều khiển tại các clock edges chính xác

2. Tránh tình trạng đua: Tránh mơ hồ trong việc xử lý thời gian bằng cách xác định khi nào các tín hiệu nên được đọc hoặc ghi tương ứng với đồng hồ.

3. Cải thiện khả năng đọc hiểu: phân tách rõ ràng các hành vi của tín hiệu dựa trên đồng hồ

4. Virtual interface

- **Virtual Interface** cho phép truy cập động vào các giao diện, tạo ra sự linh hoạt khi kết nối các thành phần khác nhau của testbench với DUT.
- Chúng đặc biệt hữu ích trong các testbench theo hướng đối tượng, nơi mà testbench có thể được tái sử dụng trong nhiều cấu hình DUT khác nhau.



Mô hình tương quan giữa interface thông thường và virtual interface

```
class test;
  virtual alu_if alu_vif;

  function new(virtual alu_if vif);
    this.alu_vif = vif;
  endfunction

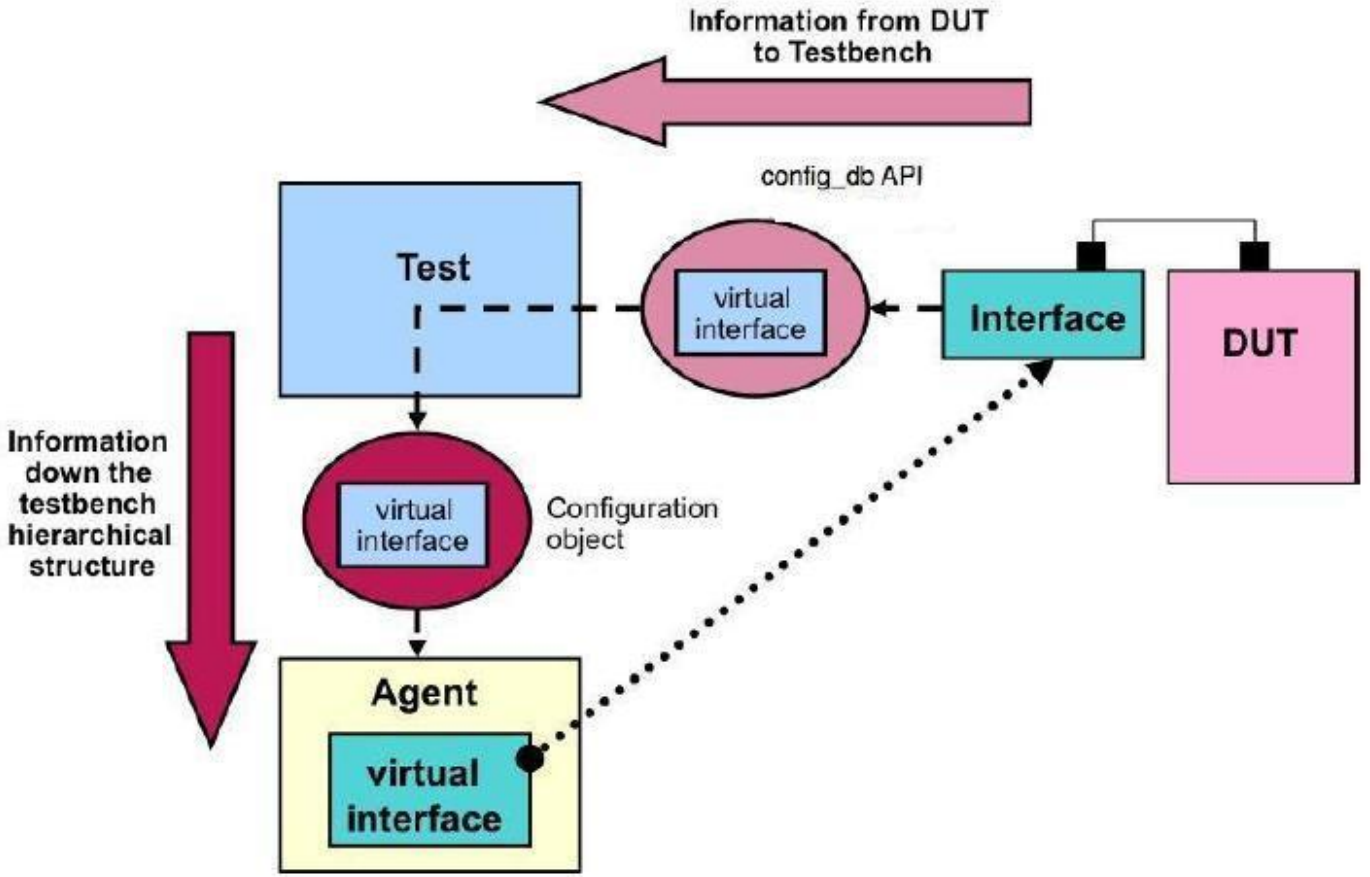
  task run_test();
    // Drive values via virtual interface
    alu_vif.src1 = 32'h1234;
    alu_vif.src2 = 32'h5678;
  endtask
endclass
```

*The virtual interface allows the test class to access the **alu_if** signals dynamically without hard-coding the interface connection.*
This is especially useful in modular, reusable testbenches.

4. Virtual interface: trường hợp áp dụng

- **Virtual Interfaces** cho phép kết nối các tác nhân khác nhau (driver, monitor, v.v.) một cách linh hoạt.
- Chúng tách biệt testbench khỏi các cấu hình DUT cụ thể, giúp dễ dàng điều chỉnh cùng một testbench cho các thiết kế khác nhau.

Phân biệt: virtual interface là "mô phỏng" interface vật lý trong phần mềm, cụ thể là trong môi trường kiểm thử, không có ý nghĩa vật lý hay kết nối vật lý.



4. Virtual interface: ví dụ

1

```
class driver;
  virtual alu_if alu_vif;

  function new(virtual alu_if
vif);
    this.alu_vif = vif;
  endfunction

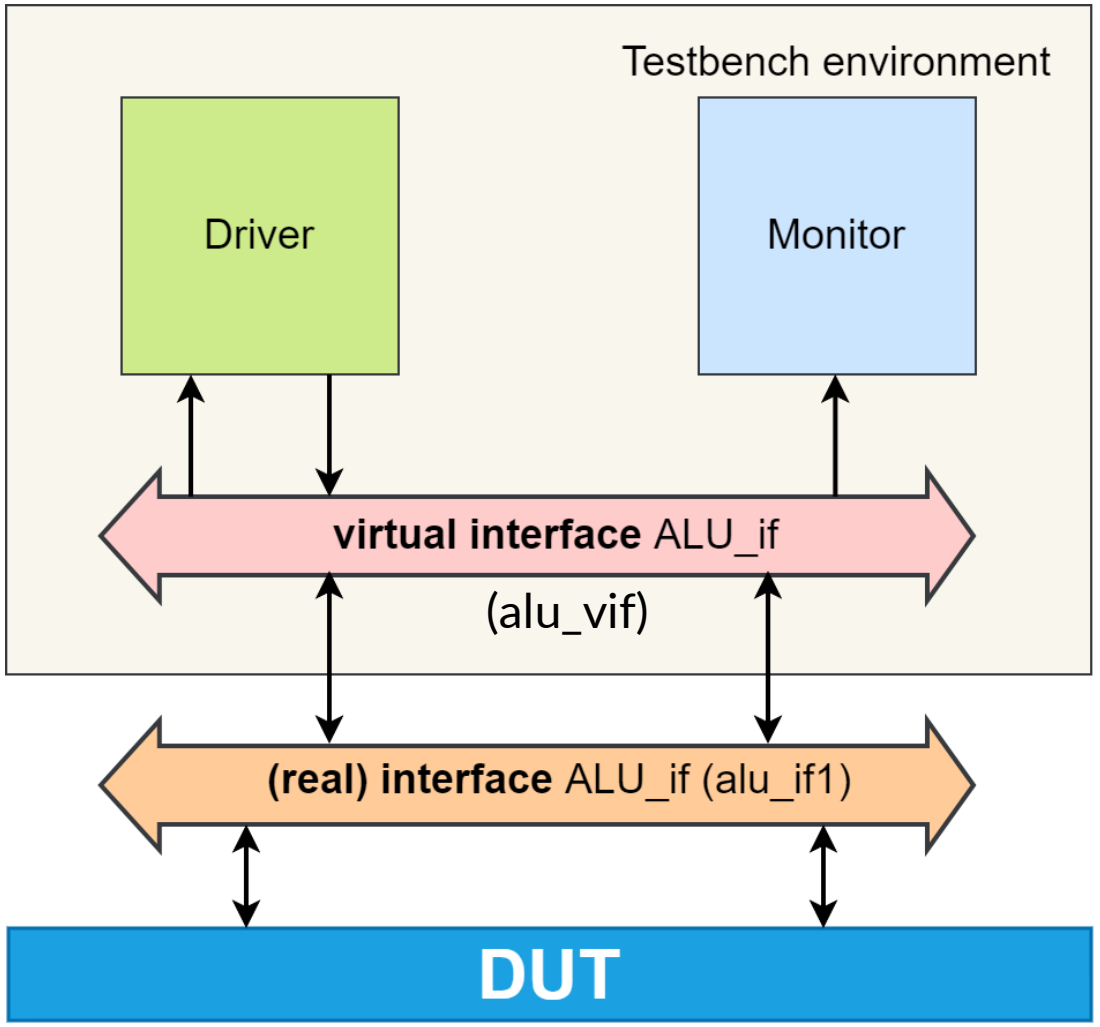
  task run();
    alu_vif.src1 = $random;
    alu_vif.src2 = $random;
    #10;
  endtask
endclass
```

2

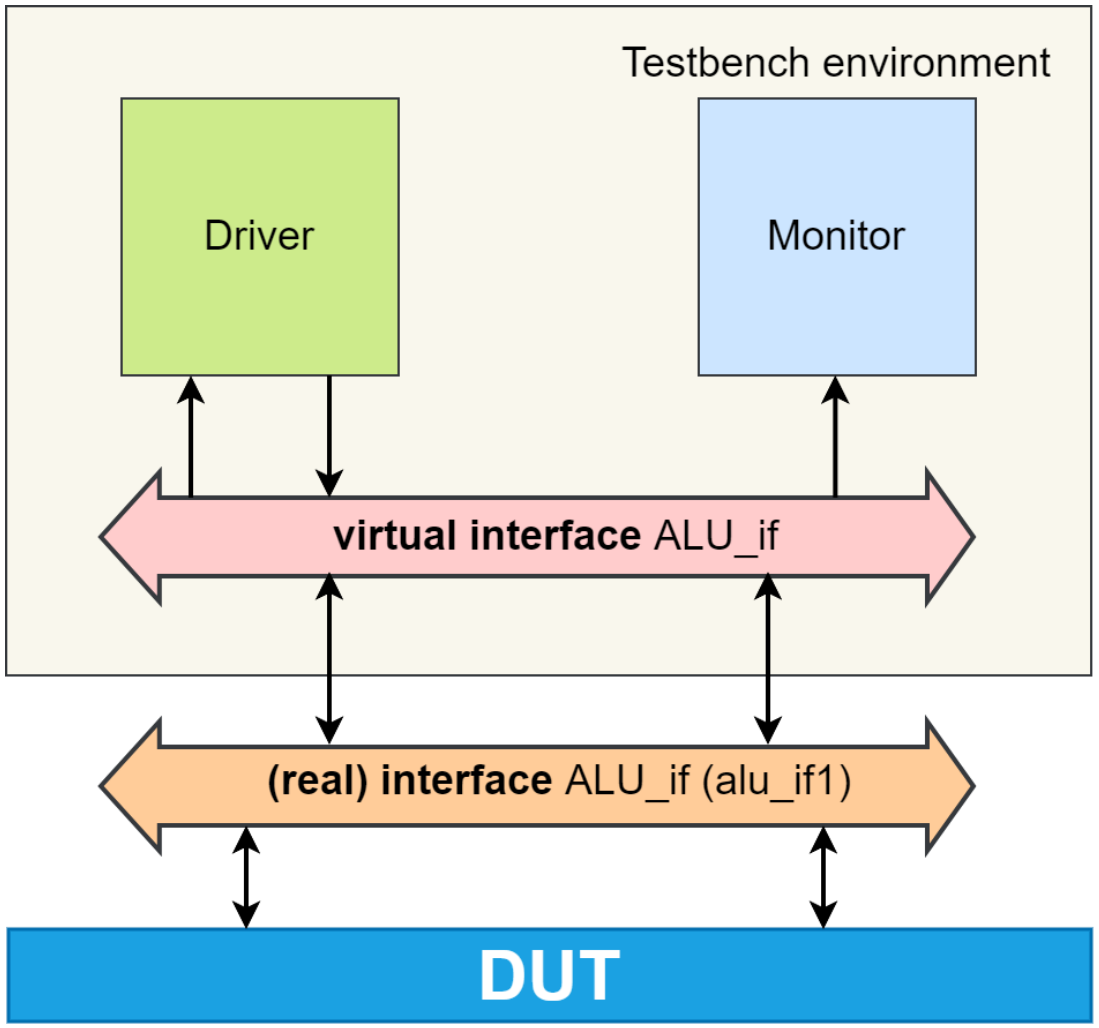
```
module top_tb;
  logic clock, reset;
  alu_if alu_if1(clock, reset);

  driver drv(alu_if1);
  monitor mon(alu_if1);

  initial begin
    drv.run();
    mon.run();
  end
endmodule
```



Giới thiệu chung về Driver và Monitor

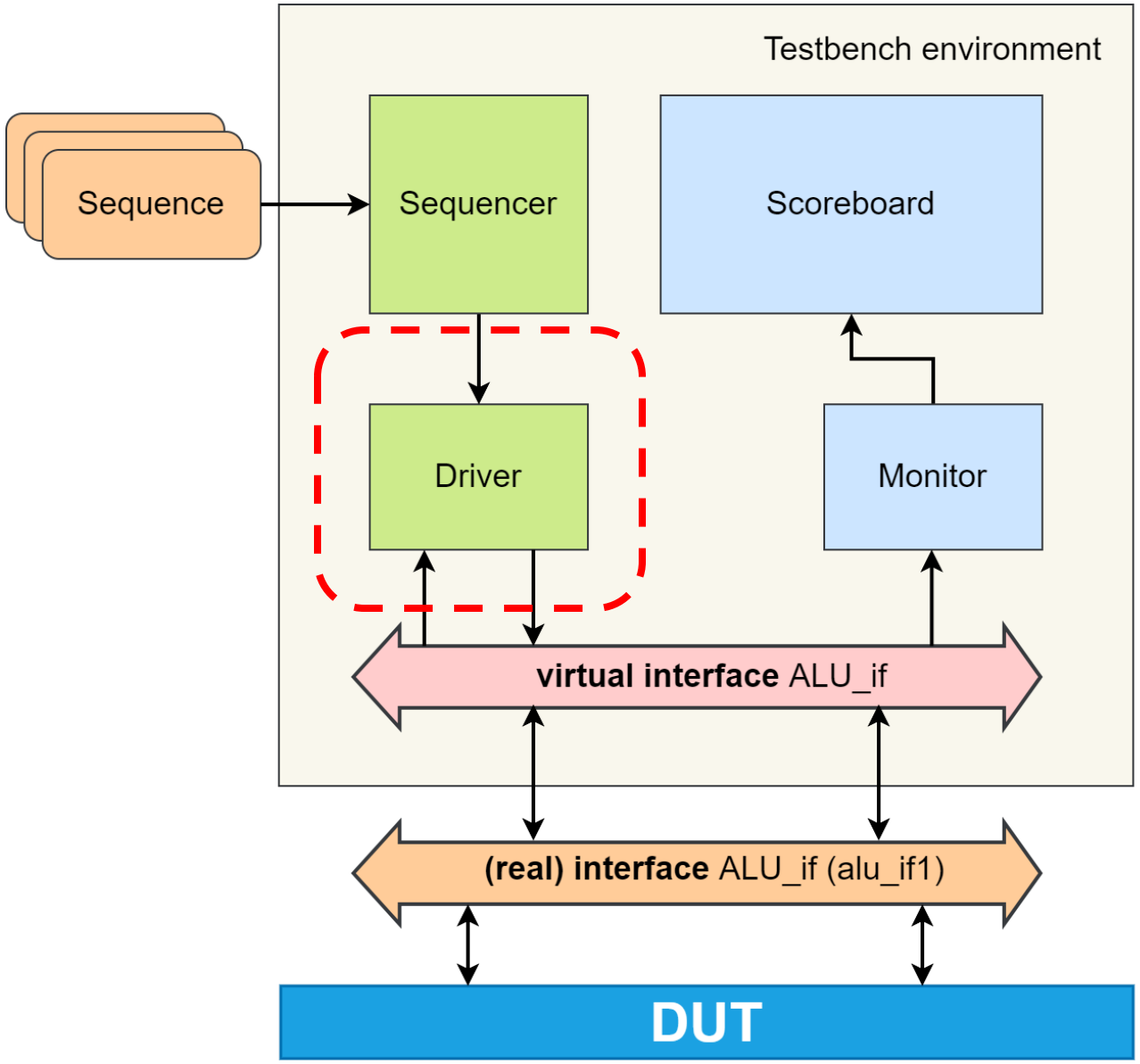


- **Driver:** Chịu trách nhiệm gửi tín hiệu kích thích (tín hiệu đầu vào) đến DUT thông qua interface.
- **Monitor:** Quan sát các tín hiệu đầu ra của DUT, ghi lại và chuyển tiếp kết quả để so sánh và phân tích.
- Cả driver và monitor đều sử dụng interface để giao tiếp với DUT, cho phép kiểm thử một cách có cấu trúc.

Driver: Ra lệnh (gửi tín hiệu) tới DUT

1. Vai trò

- Gửi các giao dịch (dữ liệu đầu vào) đến DUT.
- Lấy các giao dịch này từ sequencer hoặc chương trình kiểm thử.
- Áp dụng các giao dịch này vào DUT thông qua giao diện.
- Driver có thể được coi là bộ sinh tín hiệu kích thích, đảm nhiệm:
  - Lấy các lệnh hoặc giao dịch từ sequencer.
  - Gửi tín hiệu đầu vào đến DUT thông qua interface.
  - Chờ DUT xử lý các đầu vào trước khi gửi giao dịch tiếp theo.



Driver: Ra lệnh (gửi tín hiệu) tới DUT

2. Cấu trúc mã của Driver

- Lớp **driver** được khởi tạo trong testbench và nhận các giao dịch từ **sequencer**. Sau đó, nó gửi tín hiệu đầu vào đến **DUT** thông qua interface.

*Driver sử dụng một virtual interface (**alu_vif**) để truy cập các tín hiệu được bao bọc trong interface và đưa các giao dịch (tín hiệu đầu vào) đến DUT.*

```
class driver;
        virtual alu_if alu_vif;  // Virtual interface to interact
with the DUT

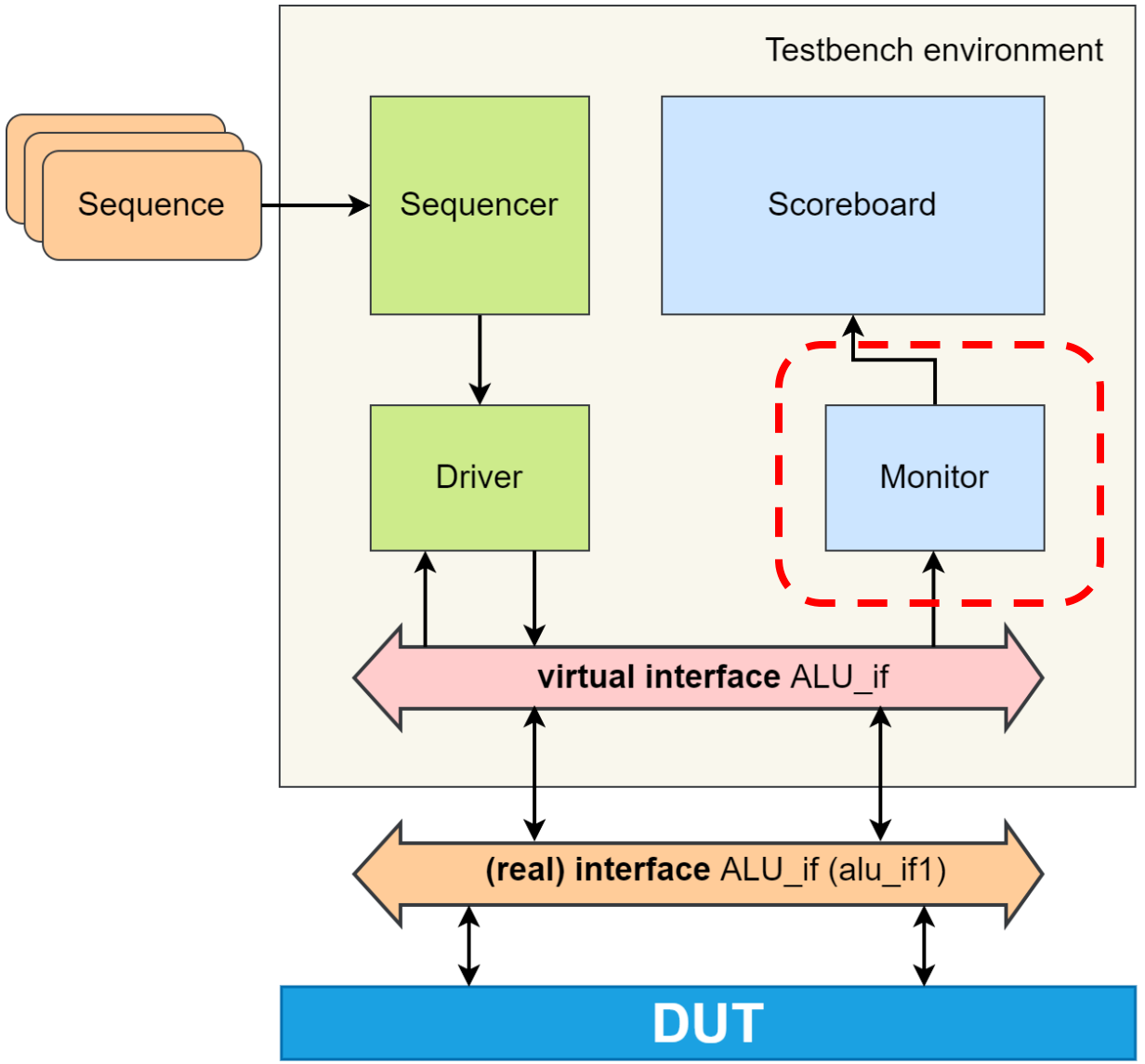
  function new(virtual alu_if vif);
        this.alu_vif = vif;
  endfunction

  task run();
     forever begin
        alu_transaction trans;
        seq.get_next_item(trans);  // Fetch transaction from
the sequencer
        alu_vif.src1 <= trans.src1;  // Drive signals to the DUT
        alu_vif.src2 <= trans.src2;
        alu_vif.imm <= trans.imm;
        alu_vif.control_in <= trans.control_in;
        #10;  // Wait for a clock cycle
     end
   endtask
endclass
```

Monitor: Nhận lệnh (nhận tín hiệu) gửi từ DUT

1. Vai trò

- Quan sát thụ động các tín hiệu đầu ra của **DUT**.
- Ghi lại đầu ra của **DUT** thông qua giao diện và đóng gói kết quả thành một giao dịch.
- Chuyển tiếp dữ liệu đã ghi lại đến **scoreboard** hoặc các thành phần khác để xác minh.
- **Monitor** đảm bảo không có tín hiệu nào bị bỏ lỡ trong quá trình kiểm thử, vì nó liên tục theo dõi các thay đổi, cụ thể:
  - Quan sát các tín hiệu đầu ra từ **DUT** thông qua interface.
  - Lưu trữ các tín hiệu này trong một cấu trúc giao dịch.
  - Chuyển tiếp giao dịch đến **scoreboard** hoặc mô hình coverage.



2. Cấu trúc mã của Monitor

- **Monitor** thụ động ghi lại các tín hiệu đầu ra và chuyển tiếp chúng dưới dạng giao dịch

Monitor *sử dụng* ***virtual interface*** *để truy cập các tín hiệu đầu ra từ* ***DUT****.*

Monitor *ghi lại kết quả vào một giao dịch và chuyển tiếp giao dịch đó đến* ***scoreboard*** *để xác minh.*

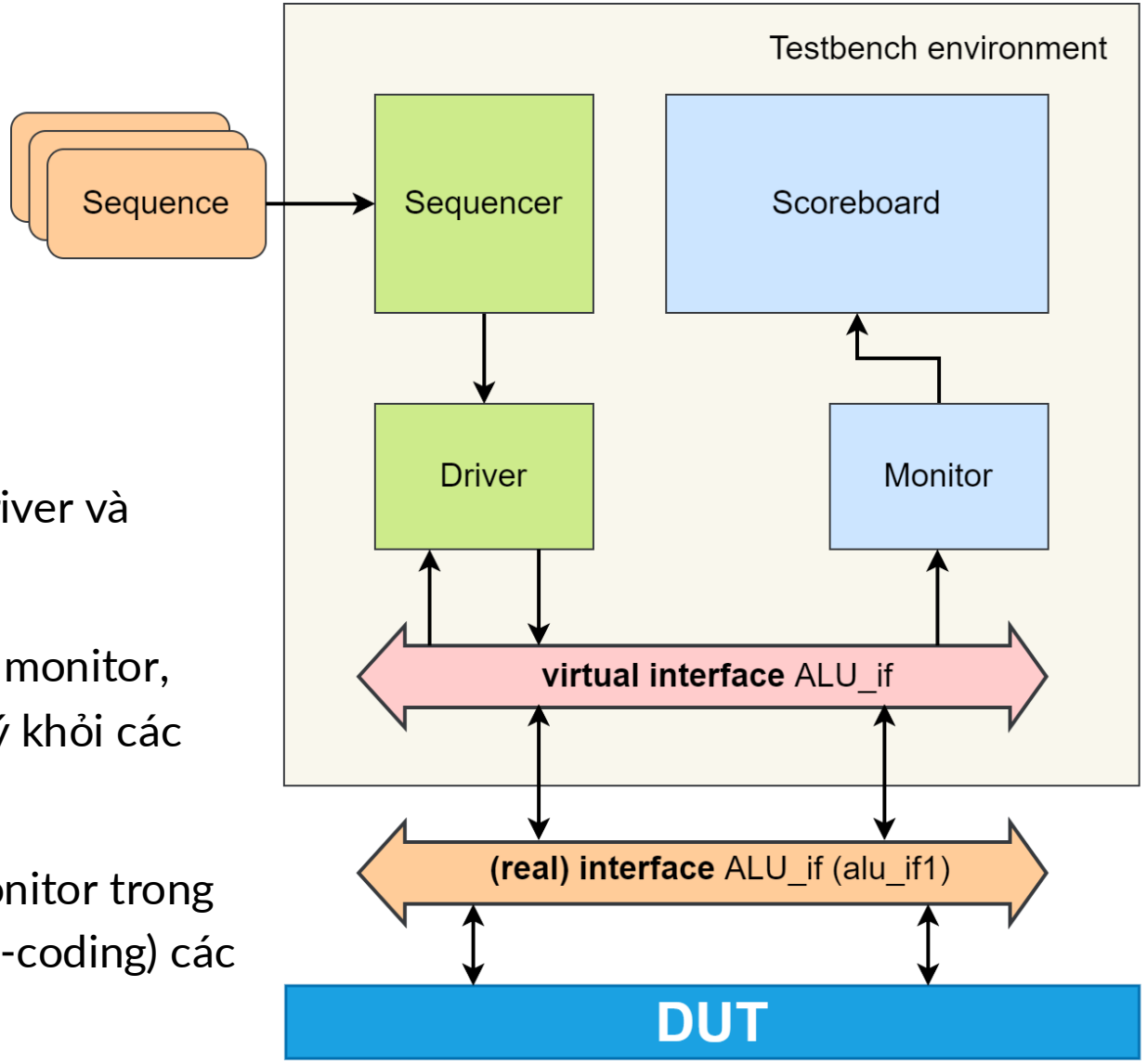
```
class monitor;
  virtual alu_if alu_vif;  // Virtual interface to interact with the
DUT
  alu_transaction trans;

  function new(virtual alu_if vif);
    this.alu_vif = vif;
  endfunction

  task run();
    forever begin
      trans = new();
      trans.aluout = alu_vif.aluout;  // Capture outputs from
the DUT
      trans.carry = alu_vif.carry;
      trans.print();  // Optionally print the captured transaction
      scoreboard.check(trans);  // Send transaction to
scoreboard for checking
      #10;
    end
  endtask
endclass
```

Cách Driver và Monitor giao tiếp qua Interface

- Driver và monitor giao tiếp với DUT thông qua interface.
- Interface bao bọc tất cả các tín hiệu cần thiết cho việc giao tiếp, đơn giản hóa kết nối giữa driver, monitor và DUT.



Virtual Interface cho Driver và Monitor:

- **Virtual interfaces** có thể linh hoạt cung cấp kết nối driver và monitor với DUT một cách động (dynamically).
- Bằng cách sử dụng **virtual interfaces** cho cả driver và monitor, chúng ta tách biệt việc instantiate một interface vật lý khỏi các triển khai lớp (không có ý nghĩa vật lý).
- Điều này cho phép tái sử dụng cùng một driver và monitor trong nhiều TB khác nhau mà không cần mã hóa cứng (hard-coding) các tên tín hiệu cụ thể.

Virtual Interface cho Driver và Monitor: ví dụ

```
class driver;
  virtual alu_if alu_vif;  // Virtual interface to interact with DUT

  function new(virtual alu_if vif);
    this.alu_vif = vif;
  endfunction
endclass

class monitor;
  virtual alu_if alu_vif;  // Virtual interface to observe DUT signals

  function new(virtual alu_if vif);
    this.alu_vif = vif;
  endfunction
endclass
```

1

```
module top_tb;
  alu_if alu_if1(clock, reset);  // Interface instance
  driver drv(alu_if1);  // Pass interface instance to driver
  monitor mon(alu_if1);  // Pass interface instance to monitor

  initial begin
    drv.run();
    mon.run();
  end
endmodule
```

2

Tương tác giữa Driver, Monitor và DUT

- Driver gửi kích thích (các lệnh hoặc giao dịch đầu vào) đến DUT.
- DUT xử lý các lệnh này và tạo ra các đầu ra.
- Monitor quan sát và ghi lại đầu ra, sau đó chuyển tiếp đến scoreboard để thực hiện so sánh với golden output.

Ứng dụng của concept driver và monitor:

Tính module hóa

Tính mở rộng

Tính tái sử dụng

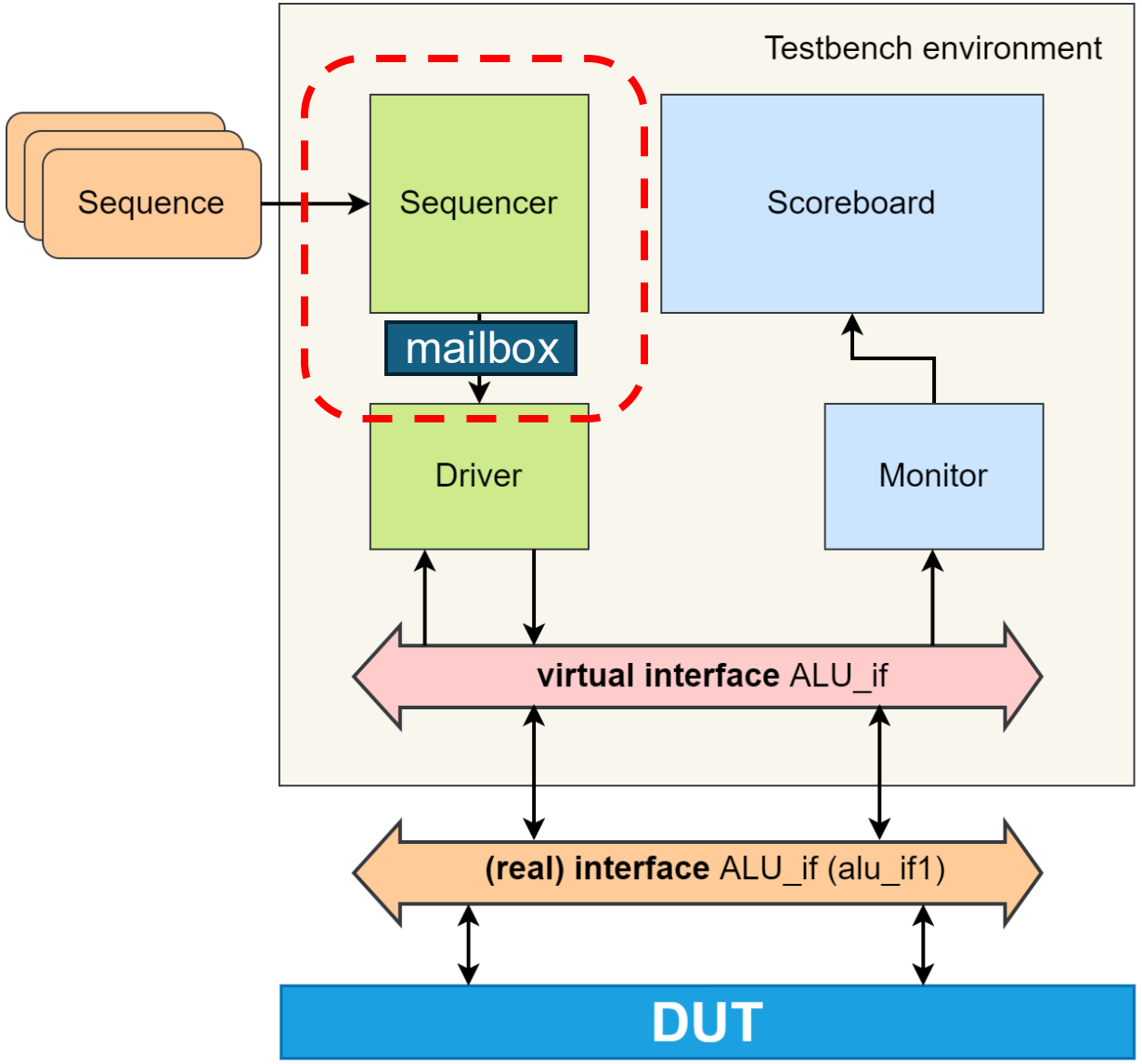
Tính bảo trì

```
// Driver sends inputs
task driver::run();
    alu_vif.src1 <= $random;
    alu_vif.src2 <= $random;
    alu_vif.control_in <= $random;
    #10;
endtask

// Monitor captures outputs
task monitor::run();
    trans.aluout = alu_vif.aluout;
    trans.carry = alu_vif.carry;
    scoreboard.check(trans);
    #10;
endtask
```

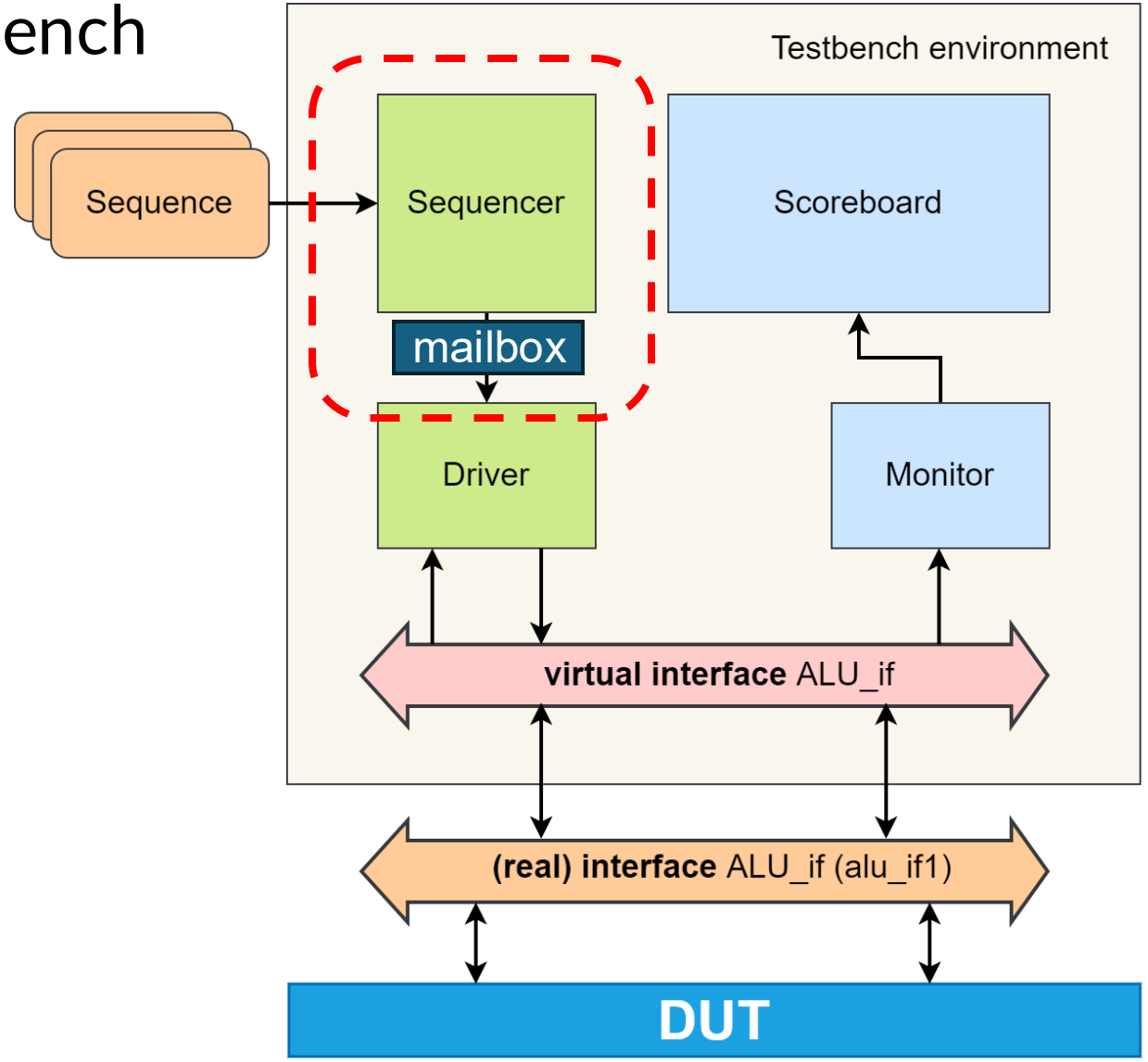
1. Tổng quan

- **Sequencer:** Một thành phần tạo và quản lý các giao dịch (hoặc gói dữ liệu) để gửi đến driver.
- **Vai trò**: Hoạt động như một bộ phát sinh kích thích (stimulus generator), tạo ra các kịch bản kiểm thử khác nhau hoặc các gói dữ liệu ngẫu nhiên.
- **Giao tiếp qua Mailbox**: Sequencer gửi các giao dịch này đến driver thông qua một mailbox, đây là cơ chế để giao tiếp liên tiến trình an toàn.

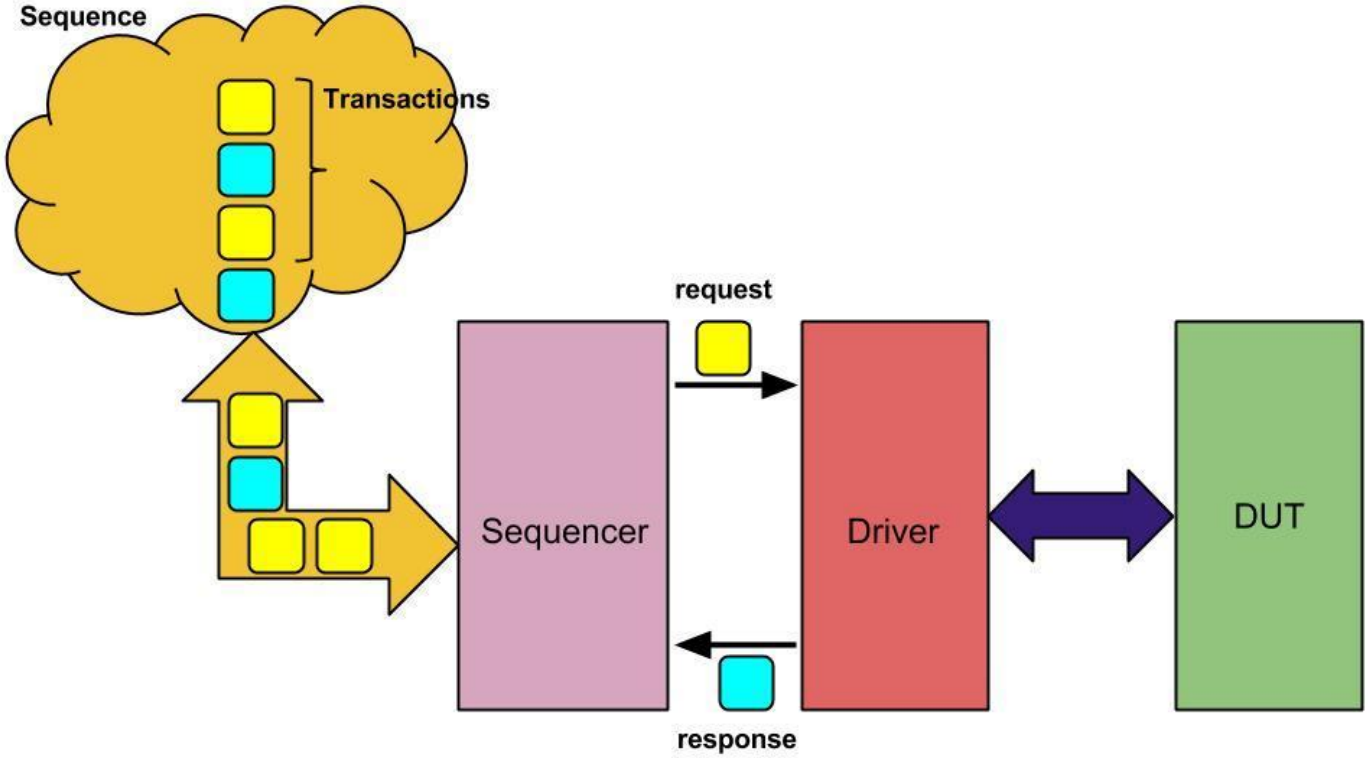


2. Vai trò của Sequencer trong testbench

- **Sequencer** đóng vai trò là bộ tạo các giao dịch, đại diện cho các đầu vào gửi đến **DUT**.
- Nó tạo ra các giao dịch và gửi chúng đến driver, driver sau đó sẽ truyền các giao dịch này đến **DUT** thông qua **interface**.
- **Sequencer** có thể tạo ra các giao dịch ngẫu nhiên hoặc các trường hợp kiểm thử đã được định trước.



Transactions/Packet



- Các giao dịch (transactions) được sử dụng để mô hình hóa quá trình giao tiếp dữ liệu.
- Transactions trong UVM đại diện cho các gói dữ liệu hoặc sự kiện được truyền qua lại giữa các thành phần xác minh, chẳng hạn như từ các trình điều khiển (drivers) đến DUT, hoặc từ DUT trở lại môi trường testbench.
- UVM cung cấp một lớp giao dịch (transaction class) có thể được mở rộng để tạo các đối tượng giao dịch mang thông tin giữa DUT và testbench.

Transactions/Packet: ví dụ

1

```
class alu_transaction;

  logic [31:0] src1, src2, imm,
mem_data_read_in;

  logic [6:0] control_in;

  function new();
    src1 = 0;
    src2 = 0;
    imm = 0;
    mem_data_read_in = 0;
    control_in = 0;

  endfunction
```

2

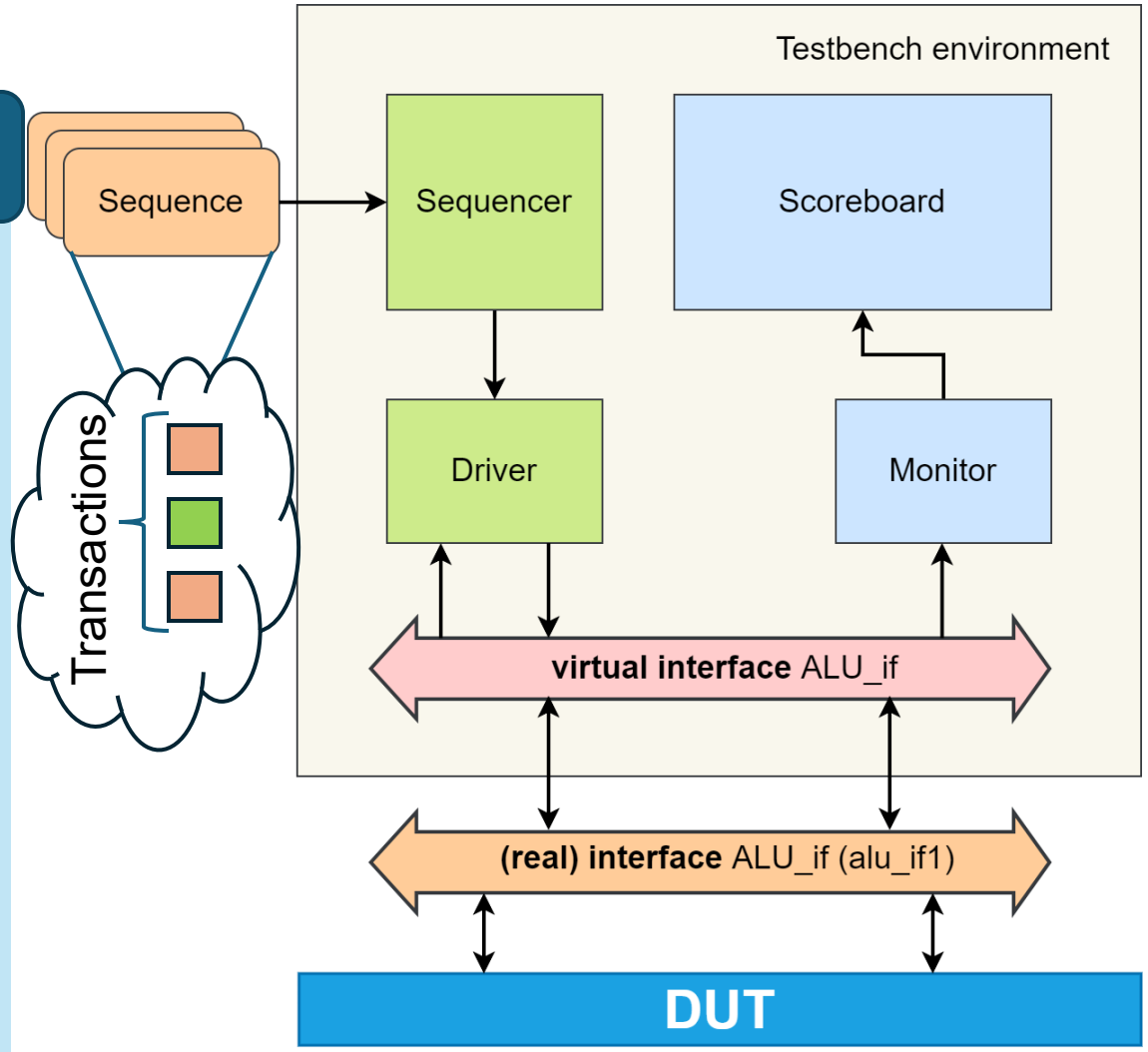
```
  function void print();

     $display("src1: %0d,
src2: %0d, imm: %0d,
control_in: %b", src1, src2,
imm, control_in);

    endfunction

endclass
```

Lớp alu_transaction bao gồm tất cả các tín hiệu đầu vào sẽ được tạo và gửi đến driver.

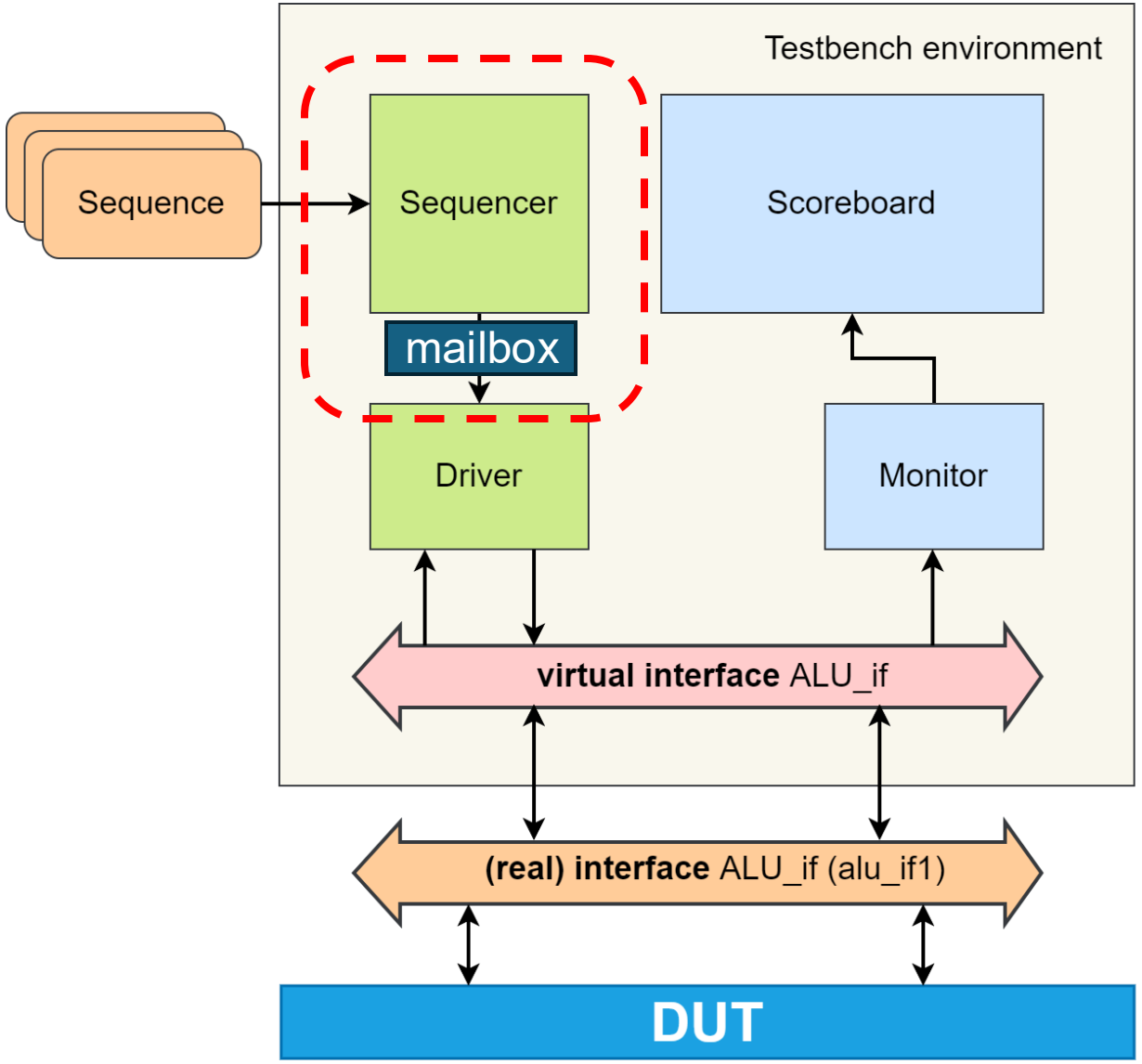


Giao tiếp giữa sequencer và driver

- Một **mailbox** được sử dụng để giao tiếp giữa sequencer và driver trong testbench.
- Nó cung cấp cơ chế **FIFO** (First In, First Out) để chuyển các giao dịch an toàn giữa các tiến trình.
- **Sequencer** đặt các giao dịch vào **mailbox**, và **driver** sẽ lấy chúng ra để áp dụng cho **DUT**.
- **Ví dụ:**

```
mailbox #(alu_transaction) trans_mbox = new();
```

➢ **Giải thích**: Mailbox cho phép sequencer và driver hoạt động song song mà không bị chặn, đảm bảo các giao dịch được truyền an toàn giữa chúng.



Tạo các Transactions trong sequencer

- **Sequencer** tạo ra các **giao dịch**, có thể là **ngẫu nhiên** hoặc **dựa trên** các giá trị định trước.
- Các giao dịch này được **ngẫu nhiên hóa** hoặc **thiết kế** dựa trên yêu cầu của các trường hợp kiểm thử cụ thể.
- Sau khi các **giao dịch** được tạo ra, chúng sẽ được gửi đến **mailbox**.
- Ví dụ:

1

```
class sequencer;

  mailbox #(alu_transaction) trans_mbox;
  function new(mailbox #(alu_transaction) mbox);
    this.trans_mbox = mbox;
  endfunction
```

2

```
 task run();
    forever begin
      alu_transaction trans = new();
      trans.src1 = $random;
      trans.src2 = $random;
      trans.imm = $random;
      trans.control_in = $random;
      trans_mbox.put(trans);  // Put the transaction into
the mailbox
      #10;  // Delay between transactions
    end
  endtask
endclass
```

➢ **Giải thích**: Sequencer tạo ra một giao dịch bằng cách ngẫu nhiên hóa các đầu vào (như src1, src2, v.v.) và sau đó đặt giao dịch này vào mailbox bằng cách sử dụng phương thức put()

Driver lấy giao dịch từ mailbox

- **Driver** lấy các giao dịch từ **mailbox** bằng cách sử dụng phương thức **get().**
- Sau khi lấy được, **driver** sẽ áp dụng các **giá trị của giao dịch** lên các tín hiệu đầu vào của DUT thông qua **interface**.
- Ví dụ:

1

```
class driver;

  mailbox #(alu_transaction) trans_mbox;
  virtual alu_if alu_vif;

  function new(mailbox #(alu_transaction) mbox,
virtual alu_if vif);

    this.trans_mbox = mbox;
    this.alu_vif = vif;
  endfunction
```

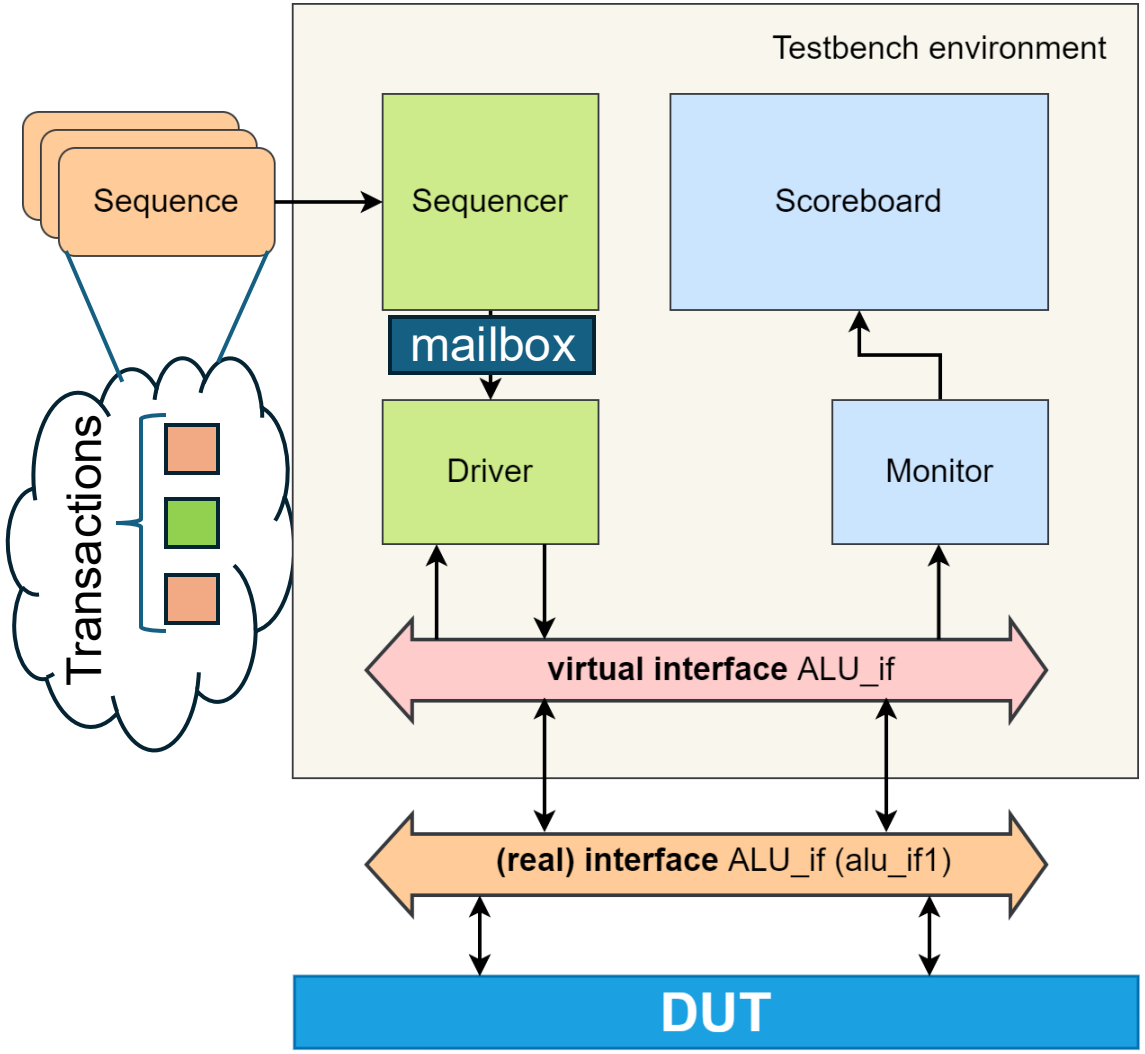
2

```
 task run();
    alu_transaction trans;
    forever begin
      trans_mbox.get(trans);  // Fetch the transaction from
the mailbox
      alu_vif.src1 <= trans.src1;
      alu_vif.src2 <= trans.src2;
      alu_vif.control_in <= trans.control_in;
      #10;  // Wait for transaction to be processed
    end
  endtask
endclass
```

➢ **Giải thích**: Driver lấy giao dịch từ mailbox bằng cách sử dụng phương thức get(), sau đó truyền các giá trị của giao dịch (src1, src2, control_in) đến DUT thông qua virtual interface.

Driver lấy giao dịch từ mailbox

Minh họa trình tự driver lấy giao dịch từ mailbox và áp dụng nó lên DUT.



Ngẫu Nhiên Hóa và Kiểm Soát Trong Driver

- Bộ điều khiển (sequencer) có thể tạo ra các giao dịch ngẫu nhiên hoặc tuân theo các kịch bản đã được định nghĩa trước tùy thuộc vào trường hợp thử nghiệm.
- Kiểm soát đối với việc ngẫu nhiên hóa có thể được áp dụng bằng cách sử dụng các ràng buộc hoặc các bài thử có định hướng để mô phỏng các kịch bản cụ thể.
- Ví dụ:

➢ ***Giải thích***: *Việc ngẫu nhiên hóa có thể được kiểm soát bằng các ràng buộc để tạo ra các kịch bản thử nghiệm cụ thể, chẳng hạn như các giá trị cụ thể cho control_in hoặc các khoảng giá trị cho src1 và src2*

```
class sequencer;

  task run();
    forever begin
      alu_transaction trans = new();
      assert(trans.randomize() with {
        src1 inside {0, 32'hFFFFFFFF};
        src2 inside {32'h0, 32'h1};
        control_in == 7'b0000001;  // Specific control
signal
      });
      trans_mbox.put(trans);
      #10;
    end
  endtask

endclass
```

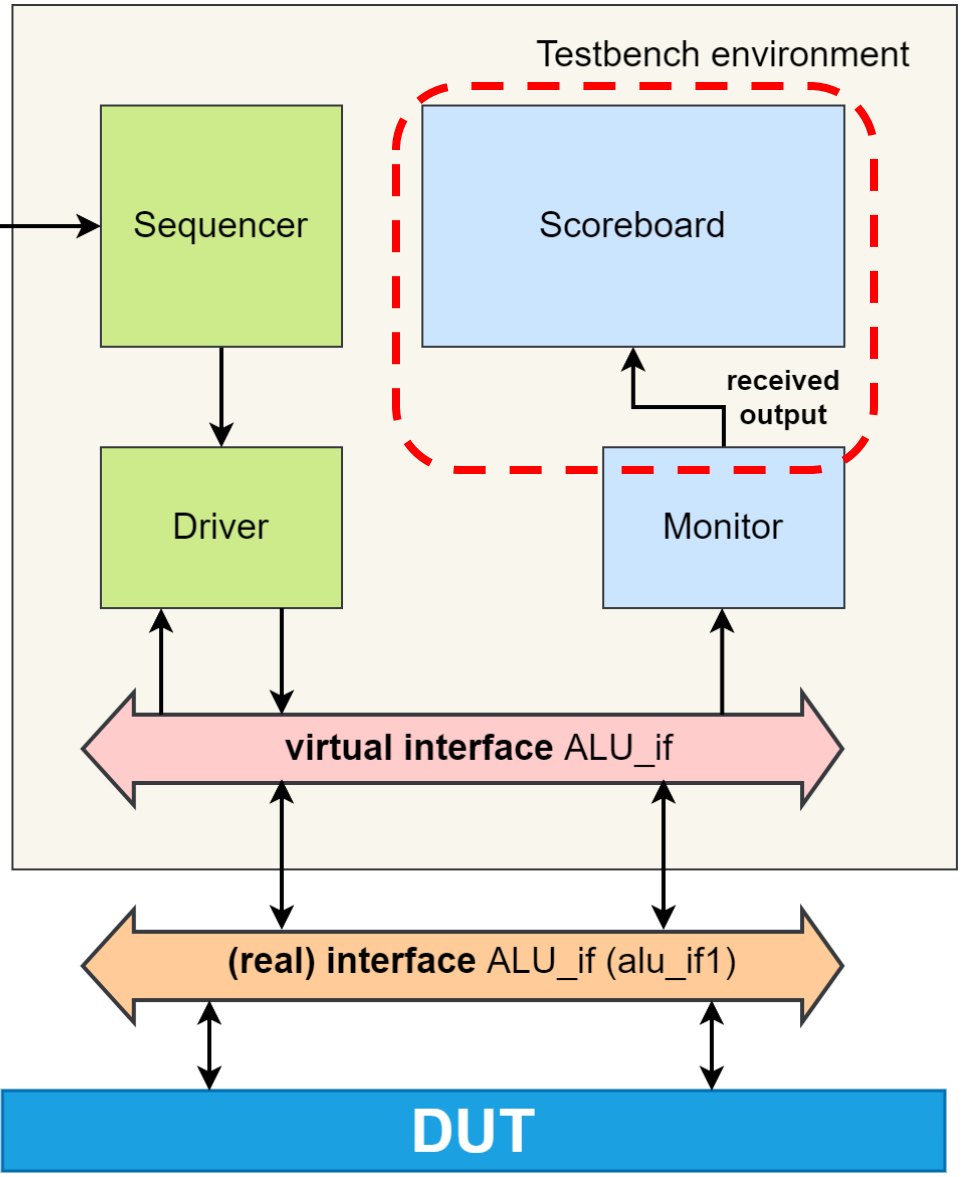
Sequencer Trong Testbenches SystemVerilog

- **Sequencer** là bộ phát sinh giao dịch cốt lõi, chịu trách nhiệm tạo ra các **test scenarios** để điều khiển DUT.
- Nó giao tiếp với **driver** thông qua **mailbox**, đảm bảo dòng chảy **transactions** liên tục.
- Bằng cách sử dụng **randomization** và **constraints**, **sequencer** có thể tạo ra nhiều **test cases** đa dạng, từ các kịch bản đơn giản đến phức tạp.
- **Mailbox** đảm bảo giao tiếp an toàn và đồng bộ hóa dòng chảy giao dịch giữa **sequencer** và **driver**.
- **Điểm nhấn**:
  - **Sequencer** cho phép tạo ra các bài thử linh hoạt, giúp dễ dàng bao quát một loạt các kịch bản.
  - **Mailbox** đảm bảo giao tiếp hiệu quả và an toàn giữa **sequencer** và **driver**.

2. Systemverilog Scoreboard

Vai trò:

- Monitor ghi lại các đầu ra từ DUT và chuyển tiếp chúng đến scoreboard.
- Scoreboard kiểm tra xem các giá trị đầu ra (received output) có khớp với kết quả mong đợi (golden output) hay không.
- Nó cung cấp phản hồi nếu có sự không khớp hoặc lỗi, giúp xác định các vấn đề trong DUT.




```
class alu_transaction;
    logic [31:0] src1, src2, imm;
    logic [6:0] control_in;
    logic [31:0] aluout;
    logic carry;
```

1

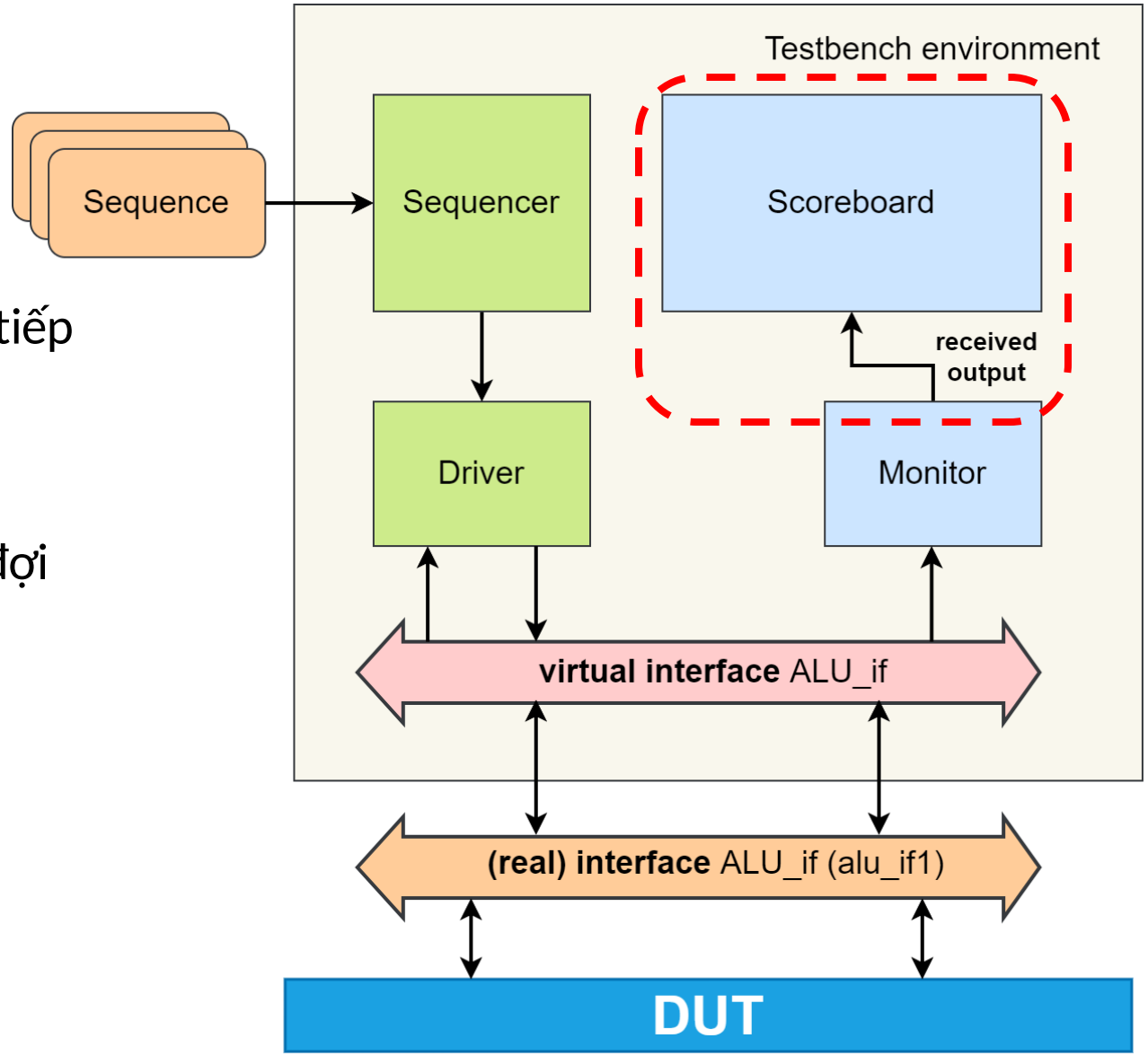
```
  function new();
    src1 = 0;
    src2 = 0;
    imm = 0;
    aluout = 0;
    carry = 0;
  endfunction
endclass
```

2

2. Systemverilog Scoreboard

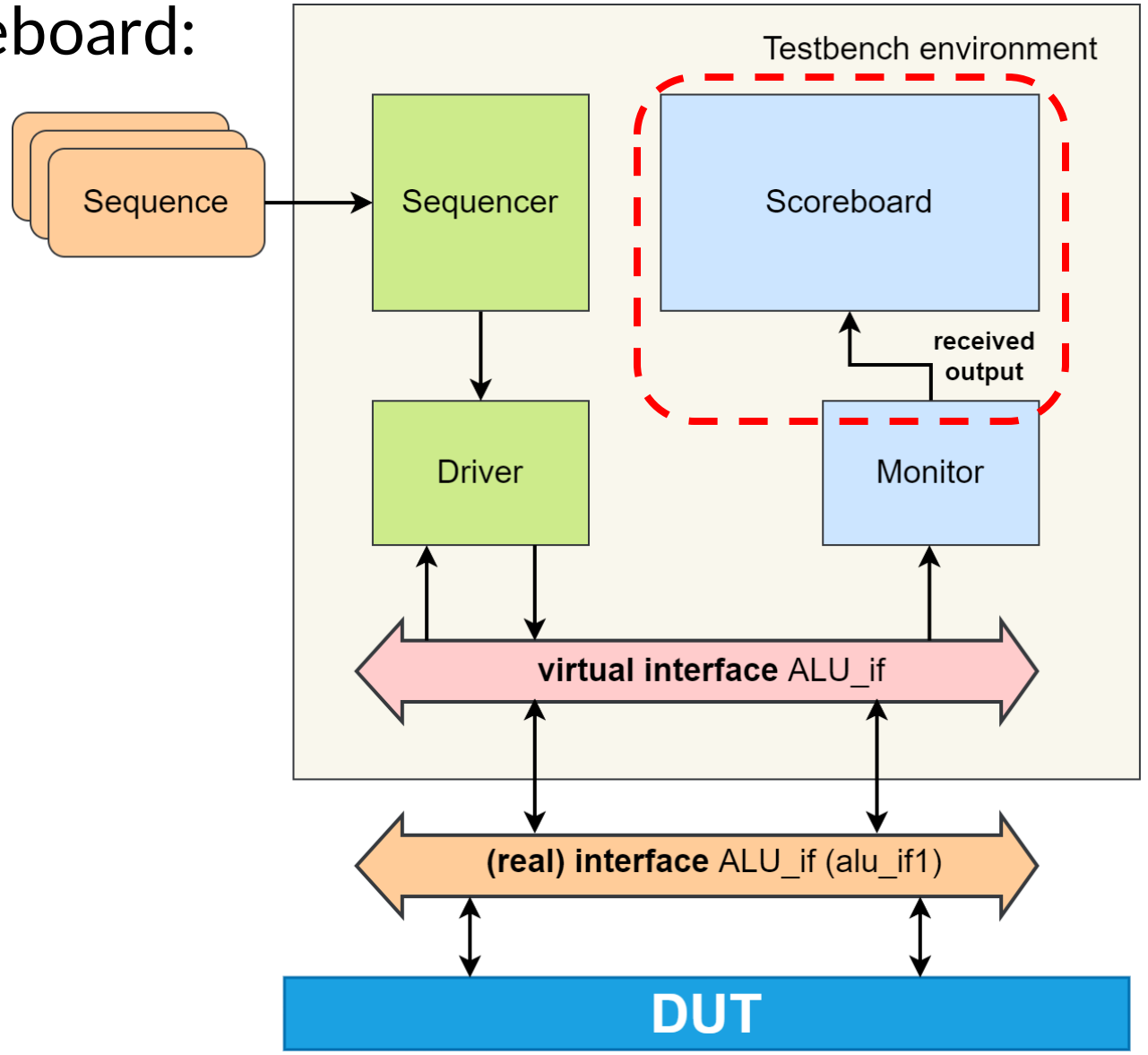
Vai trò của Scoreboard:

- Monitor ghi lại các đầu ra từ DUT và chuyển tiếp chúng đến scoreboard.
- Scoreboard kiểm tra xem các giá trị đầu ra (received output) có khớp với kết quả mong đợi (golden output) hay không.
- Nó cung cấp phản hồi nếu có sự không khớp hoặc lỗi, giúp xác định các vấn đề trong DUT.



Transactions và Packets trong Scoreboard:

- Monitor tạo ra các giao dịch (hoặc gói tin) chứa các giá trị đầu ra từ DUT.
- Các giao dịch này được gửi đến scoreboard để xác minh.
- Scoreboard kiểm tra từng giao dịch, so sánh nó với các giá trị mong đợi để xác nhận tính chính xác.



Transactions và Packets trong Scoreboard:

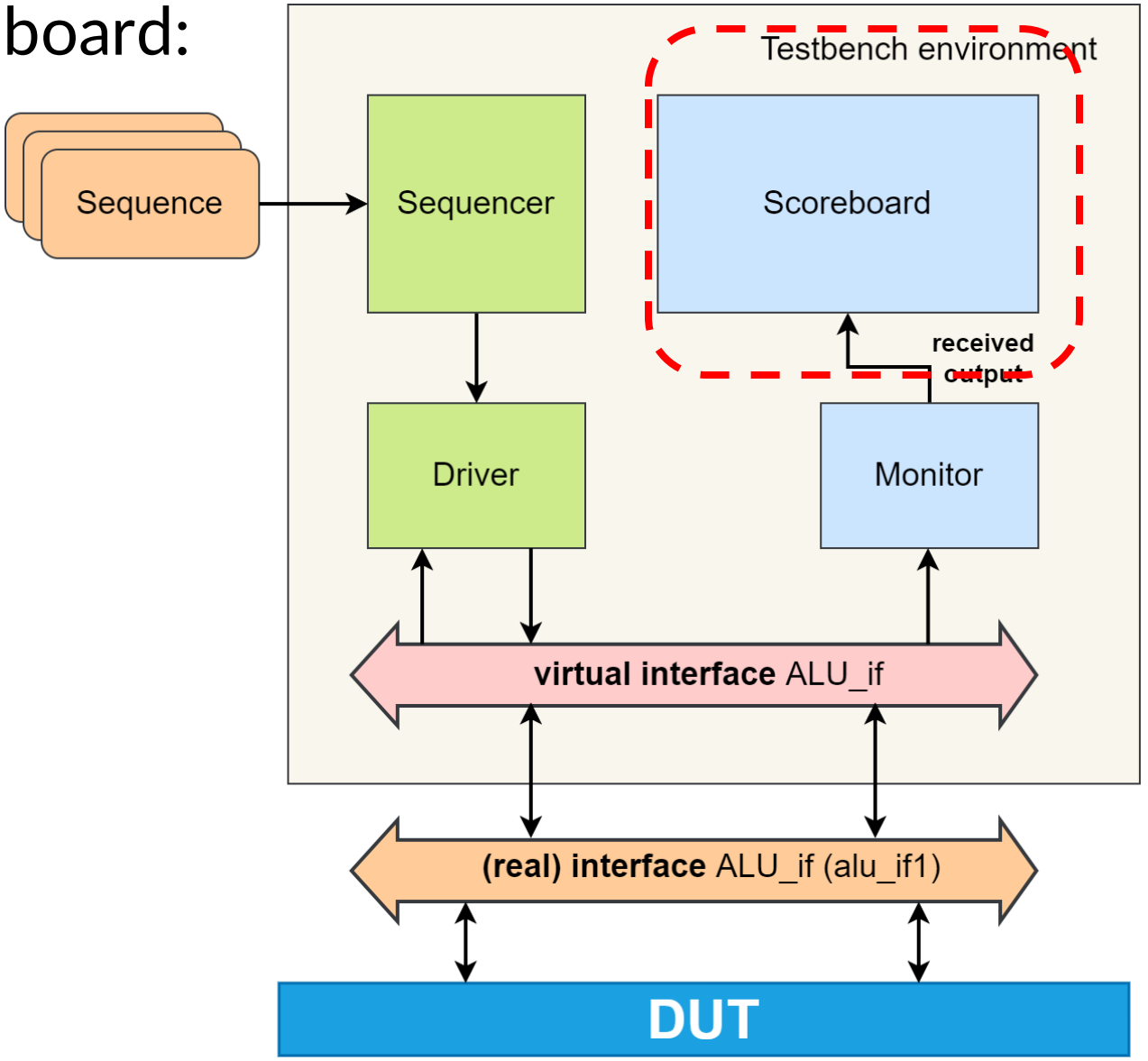
- Ví dụ

```
class alu_transaction;

  logic [31:0] src1, src2, imm;
  logic [6:0] control_in;
  logic [31:0] aluout;
  logic carry;

  function new();
    src1 = 0;
    src2 = 0;
    imm = 0;
    aluout = 0;
    carry = 0;
  endfunction

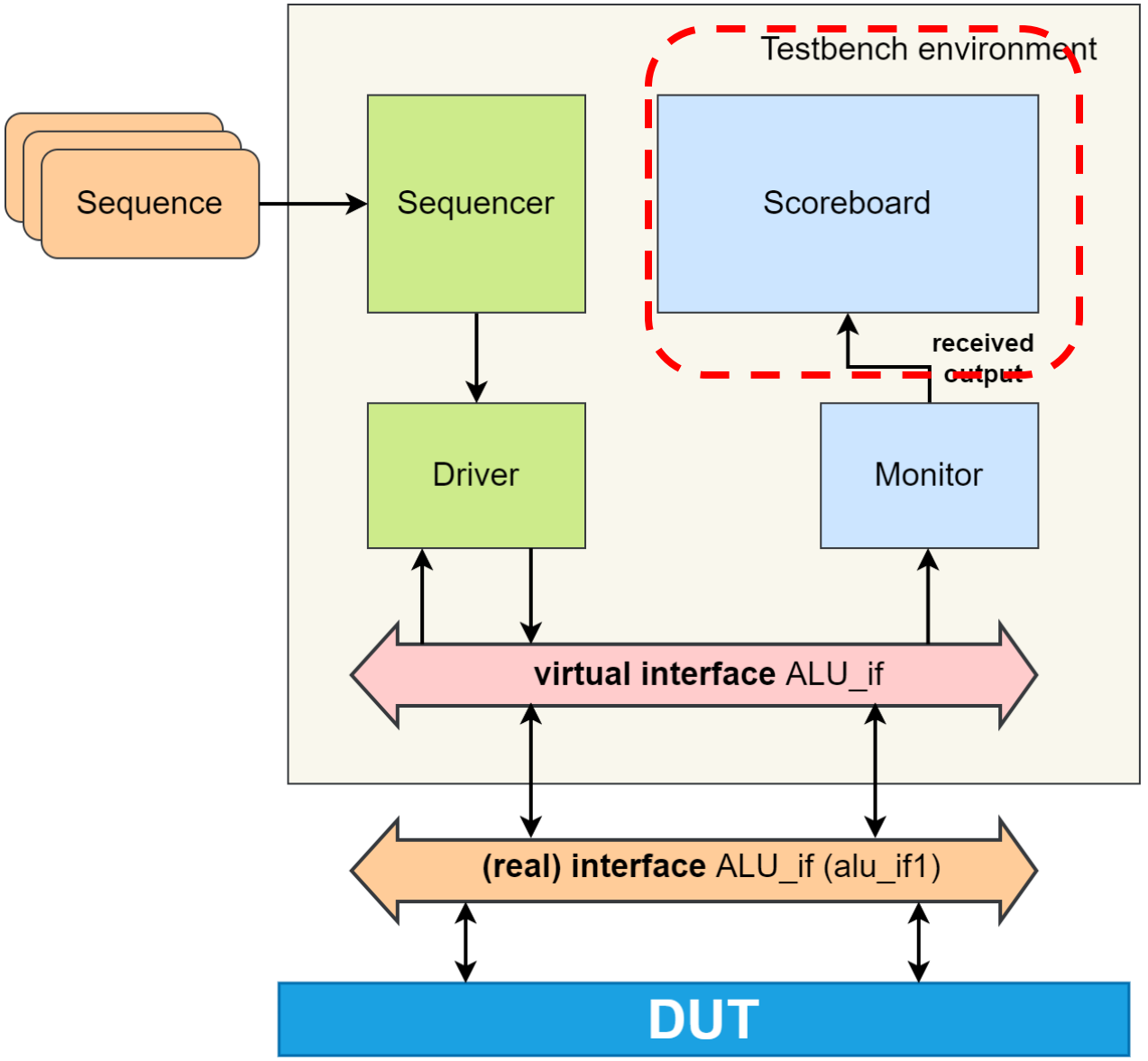
endclass
```



2. Systemverilog Scoreboard

Cách Scoreboard hoạt động:

- **Scoreboard** nhận các giao dịch từ **monitor** chứa các đầu ra thực tế (aluout, carry).
- Nó so sánh các đầu ra thực tế với các đầu ra mong đợi, được tạo ra dựa trên các đầu vào hoặc một mô hình tham chiếu vàng (**golden reference model**).
- Nếu phát hiện sự không khớp, **scoreboard** sẽ tạo ra một thông báo **lỗi**, giúp xác định các vấn đề trong chức năng của DUT.



Cách Scoreboard hoạt động:

- Ví dụ:

➢ Phương thức check so sánh các đầu ra thực tế (aluout và carry) từ giao dịch với các giá trị mong đợi.Nếu các đầu ra không khớp, scoreboard sẽ phát sinh lỗi."

1

```
class scoreboard;

  function new();
  endfunction

  task check(alu_transaction trans);
    logic [31:0] expected_aluout;
    logic expected_carry;
    // Example: Expected output for addition
    expected_aluout = trans.src1 + trans.src2;
    expected_carry = (trans.src1 + trans.src2) > 32'hFFFFFFFF;
  // Example: Expected output for addition
    expected_aluout = trans.src1 + trans.src2;
    expected_carry = (trans.src1 + trans.src2) > 32'hFFFFFFFF;
```

2

```
  // Compare actual output with expected
    if (trans.aluout !== expected_aluout ||
        trans.carry !== expected_carry) begin

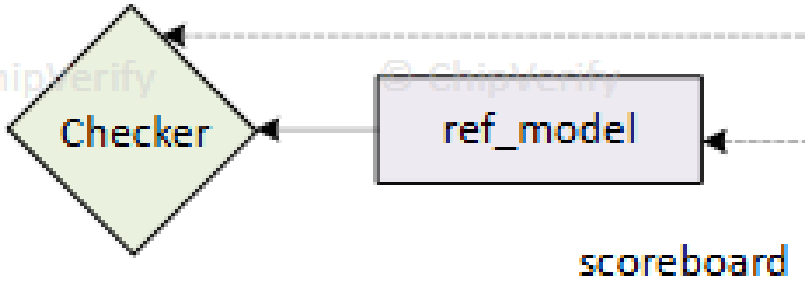
      $error("Mismatch: Expected aluout = %0d,
carry = %b, but got aluout = %0d, carry = %b",

            expected_aluout, expected_carry,
trans.aluout, trans.carry);

    end
  endtask
endclass
```

2. Systemverilog Scoreboard

- Tạo Ra Các Đầu Ra Mong Đợi Trong Scoreboard
  - Scoreboard cần tính toán các đầu ra mong đợi dựa trên các giá trị đầu vào
  - Ví dụ, nếu thao tác của ALU là phép cộng, scoreboard sẽ tính expected_aluout = src1 + src2.
  - Đối với các thao tác phức tạp hơn, scoreboard có thể sử dụng mô hình tham chiếu vàng (golden reference model) hoặc logic để xác định đầu ra mong đợi chính xác.




```
task generate_expected_output(alu_transaction trans,
            output logic [31:0] expected_aluout,
            output logic expected_carry);

  case (trans.control_in[6:4])

    3'b000: begin  // ADD operation

      expected_aluout = trans.src1 + trans.src2;

      expected_carry = (trans.src1 + trans.src2) > 32'hFFFFFFFF;
    end
    // Other operations...
  endcase
endtask
```

2. Systemverilog Scoreboard

- Xử lý sự không khớp và lỗi trong Scoreboard
  - Nếu đầu ra thực tế từ DUT không khớp với đầu ra mong đợi, scoreboard sẽ phát sinh thông báo lỗi hoặc thông báo sự không khớp.
  - Scoreboard cũng cung cấp thông tin chi tiết về test case (các giá trị đầu vào, đầu ra thực tế so với đầu ra mong đợi) để hỗ trợ gỡ lỗi cho DUT.

```
if (trans.aluout !== expected_aluout || trans.carry !== expected_carry) begin

  $error("Mismatch: Expected aluout = %0d, carry = %b, but got aluout = %0d, carry = %b",
         expected_aluout, expected_carry, trans.aluout, trans.carry);

end else begin

  $display("Test Passed: aluout = %0d, carry = %b", trans.aluout, trans.carry);

end
```

➢ Giải thích: Scoreboard cung cấp phản hồi chi tiết khi có sự không khớp, giúp các kỹ sư nhanh chóng xác định các vấn đề trong logic của DUT.

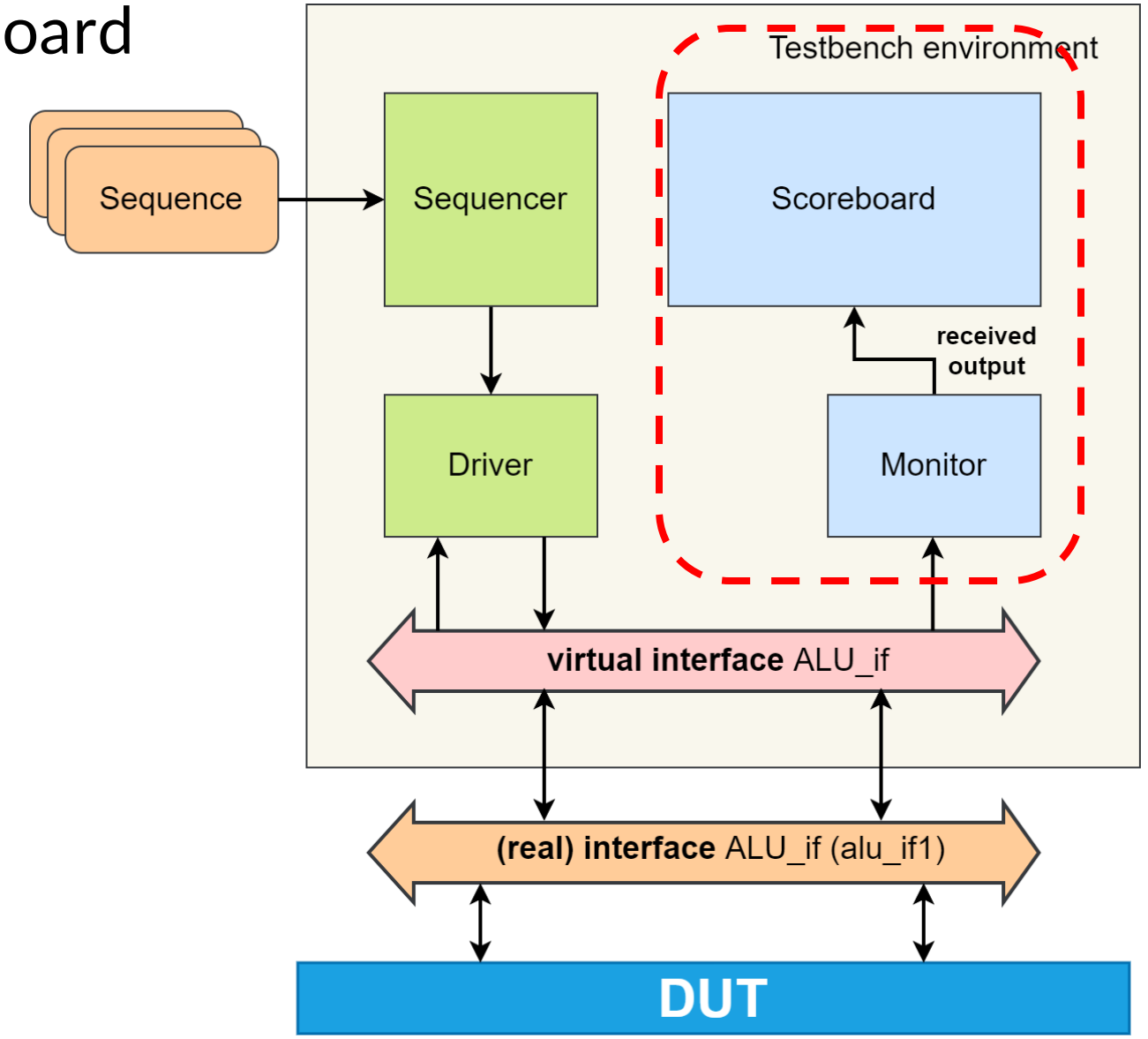
2. Systemverilog Scoreboard

- Scoreboards có thể được mở rộng để hỗ trợ các tính năng nâng cao hơn:
  - **Coverage Collection**: Theo dõi số lượng các test cases thành công hoặc thất bại để đảm bảo tất cả các kịch bản đều được thử nghiệm.
  - **Multi-threaded Checking**: Xử lý nhiều giao dịch đồng thời cho các thiết kế phức tạp.
  - **Golden Reference Model Comparison**: So sánh các đầu ra của DUT với một mô hình tham chiếu vàng để có độ chính xác cao hơn trong các thiết kế phức tạp.

➢Đối với các thiết kế phức tạp, scoreboards có thể so sánh các đầu ra của DUT với một mô hình tham chiếu vàng, đảm bảo rằng ngay cả các trường hợp biên cũng được xử lý chính xác

Tương Tác Giữa Monitor và Scoreboard

- **Monitor** thu nhận đầu ra từ **DUT** và gửi một giao dịch đến **scoreboard**.
- **Scoreboard** tính toán đầu ra mong đợi và so sánh nó với đầu ra thực tế.
- Nếu các đầu ra khớp nhau, **test case** sẽ được coi là thành công. Nếu không, **scoreboard** sẽ phát sinh lỗi.



Tương Tác Giữa Monitor và Scoreboard: **ví dụ**

```
class monitor;

  virtual alu_if alu_vif;
  mailbox #(alu_transaction) trans_mbox;

  task run();
    forever begin
      alu_transaction trans = new();
      trans.aluout = alu_vif.aluout;
      trans.carry = alu_vif.carry;
// Send transaction to scoreboard
      trans_mbox.put(trans);
    #10;
    end
  endtask
endclass
```

1

```
class scoreboard;

  task check(alu_transaction trans);
    logic [31:0] expected_aluout;
    logic expected_carry;

  generate_expected_output(trans, expected_aluout, expected_carry);

  if (trans.aluout !== expected_aluout || trans.carry !== expected_carry)
begin

      $error("Mismatch: Expected aluout = %0d, carry = %b, but got
aluout = %0d, carry = %b",
            expected_aluout, expected_carry, trans.aluout, trans.carry);
    end
  endtask
endclass
```

2

Vai Trò Của Scoreboard

- **Scoreboard** đóng vai trò quan trọng trong việc xác minh tính đúng đắn của DUT trong bất kỳ testbench nào.
- Nó đảm bảo rằng các đầu ra của DUT khớp với kết quả mong đợi bằng cách so sánh từng giao dịch.
- Các tính năng nâng cao như **mô hình tham chiếu vàng** và **thu thập độ phủ** (coverage collection) khiến **scoreboard** trở thành một **công cụ mạnh mẽ** để xác minh.
- **Điểm Nhấn**:
  - Scoreboard kiểm tra từng giao dịch, giúp **xác định các lỗi thiết kế**.
  - Nó **cung cấp phản hồi chi tiết**, cho phép các kỹ sư **nhanh chóng gỡ lỗi các vấn đề** trong DUT.

HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.